
NI LabVIEW and Third-Party Software & Hardware Integration

Developed by UR TP team

2024-12-12

Content

	page
1. NI LabVIEW and Third-Party Software Integration	3
1.1. NI LabVIEW and MATLAB Integration	3
1.2. Exercise 1-1: Simplifying Equations with MATLAB in NI LabVIEW	6
1.3. NI LabVIEW and Python Integration	9
1.4. Exercise 1-2: Calling Python Objects from NI LabVIEW	13
1.5. Calling Shared Libraries in NI LabVIEW	16
1.6. Exercise 1-3: Building and Importing a C++ Dynamic Library in NI LabVIEW	22
2. NI LabVIEW and Third-Party Hardware Integration	32
2.1. Automating Non-NI Instruments: NI-VISA	32
2.2. Exercise 2-1: Controlling and Communicating with External Devices via Serial Connection Using NI-VISA	37
2.3. Using Raspberry Pi or Arduino with NI LabVIEW	40

1. NI LabVIEW and Third-Party Software Integration

1.1 NI LabVIEW and MATLAB Integration

In this section, you will learn the methods for integrating external programming languages and libraries into NI LabVIEW, including MATLAB scripts, Python scripts, and C/C++ DLLs, to leverage advanced functionalities and expand the capabilities of NI LabVIEW applications.

MATLAB is widely used in academic research to provide solutions for robotics, control systems, autonomous navigation, RF, and many other fields. MATLAB has many helpful functions and add-ons that can integrate with the NI LabVIEW environment, expanding NI LabVIEW's capabilities.



Note – this functionality was introduced in NI LabVIEW 2021 and is not available in previous versions. If you want to use MATLAB in older versions of NI LabVIEW, refer to the MATLAB Script or MathScript Node.

System Requirements

To use MATLAB palette, you need to have MATLAB 6.5 or later and NI LabVIEW 2021 or later installed and licensed.

NI LabVIEW MATLAB Palette

MATLAB palette provides tools to open and close MATLAB sessions and run MATLAB script through the Call MATLAB Function subVI. These functions enable seamless integration of MATLAB scripts into NI LabVIEW projects.

Function	Terminal	Description
Open MATLAB Session		This subVI opens a MATLAB session with the specified release of MATLAB. (For example, 'R2024b') If no release is specified, the most recent installed release of MATLAB is used.
Call MATLAB Function		The Call MATLAB Function subVI serves as a bridge between NI LabVIEW and MATLAB functionalities. The Call MATLAB Function calls the top-level MATLAB function from a source file. You can configure the Call MATLAB Function to specify the MATLAB session, source path, and function name. You can call both .m and .mlx files.
Close MATLAB Session		Close MATLAB Session subVI terminates the MATLAB session



Note The Call MATLAB Function is not supported on real-time or FPGA targets.



Tip The Call MATLAB Function input parameter specifies the input parameters of the MATLAB function. You must wire the inputs in the order that the function calls them. You can resize the MATLAB Node to add more terminals corresponding to the number of input arguments for your MATLAB function.



Tip When calling a MATLAB function with Call MATLAB Function subVI, you must wire a constant or control that matches the data type of the top-level MATLAB function's return type. See the following section for supported data types.

Supported Data Types

The Call MATLAB Function supports the following data types:

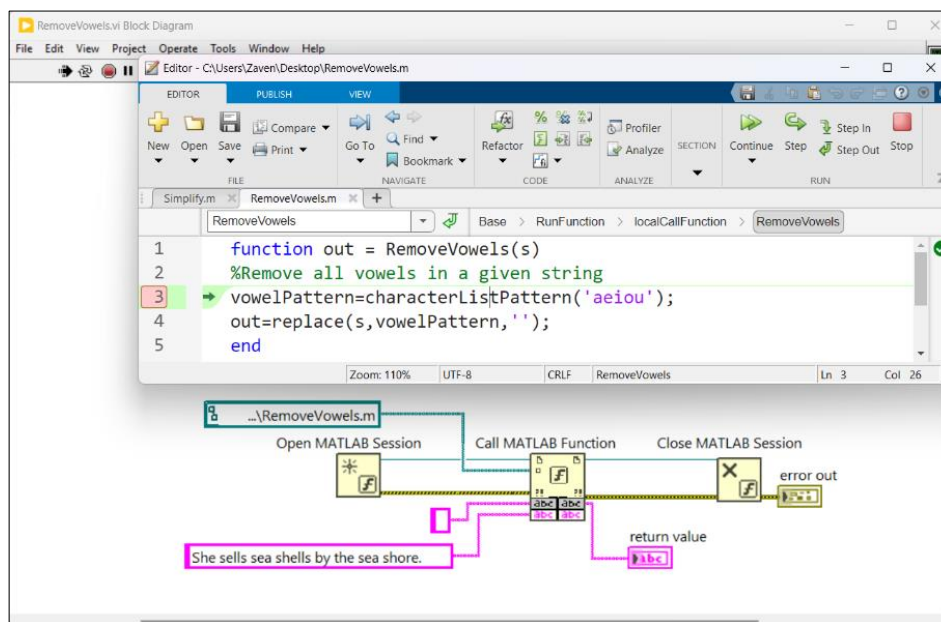
- Numerics
- Arrays, including multi-dimensional arrays.
- Strings
- Clusters
- Booleans



Note 1D arrays are converted to Row Vectors in MATLAB.

Debugging Capabilities

You can add breakpoints in your MATLAB editor, debug your code by single stepping from NI LabVIEW into your .m or .mlx file, and launch your MATLAB Function Editor from NI LabVIEW to edit your .m file on the go.



Note Debugging capabilities are available starting with NI LabVIEW 2022 Q3.

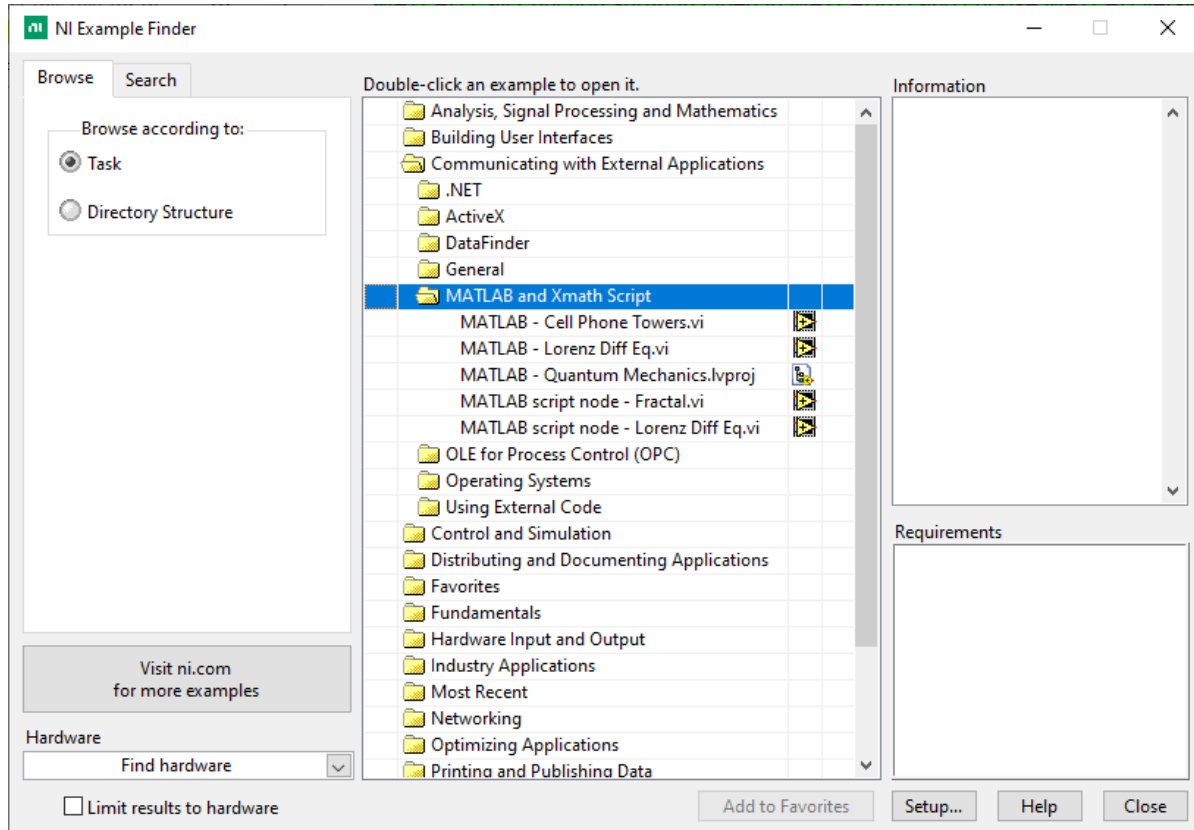


Tip To open MATLAB Function Editor directly from the NI LabVIEW you need to right click on MATLAB Call Function subVI in Open in MATLAB(R) and choose MATLAB version.

NI LabVIEW MATLAB Functions Examples

There are multiple examples from NI LabVIEW that demonstrate how to set up your block diagram and call your MATLAB functions.

Examples for using the Call MATLAB Function are available in the NI LabVIEW Example Finder (found in Help > > Find Examples) under Communicating with External Applications > > MATLAB and Xmath Script.



1.2.Exercise 1-1: Simplifying Equations with MATLAB in NI LabVIEW

Goal

Simplify mathematical equations using MATLAB scripts integrated in NI LabVIEW.

Description

You must develop an application that integrates MATLAB scripts into NI LabVIEW to automate the simplification of symbolic equations. The system must take user-input equations as strings, process them through MATLAB to reduce complexity, and display the simplified results in NI LabVIEW.



Note To complete this exercise, you must have a licensed version of MATLAB.

1. Create a blank project and save it as Exercises\Exercise 19-1 Simplifying Equations with MATLAB in NI LabVIEW \Simplify Mathematical Equations.lvproj.

2. Open MATLAB and create a new function.

- ☐ Launch MATLAB from your computer.
- ☐ In the MATLAB Home tab, click New and select Function from the dropdown menu.
- ☐ Replace the default template code in the function file with the following:

```
function out = Simplify(input_eq)
    % Simplify - Simplifies a symbolic equation string input.
    % Convert the input string to a symbolic expression
    eq = str2sym(input_eq);
    % Simplify the symbolic expression
    simp_eq = simplify(eq, "IgnoreAnalyticConstraints", true);
    % Convert the symbolic expression back to a string
    out = char(simp_eq);
end
```

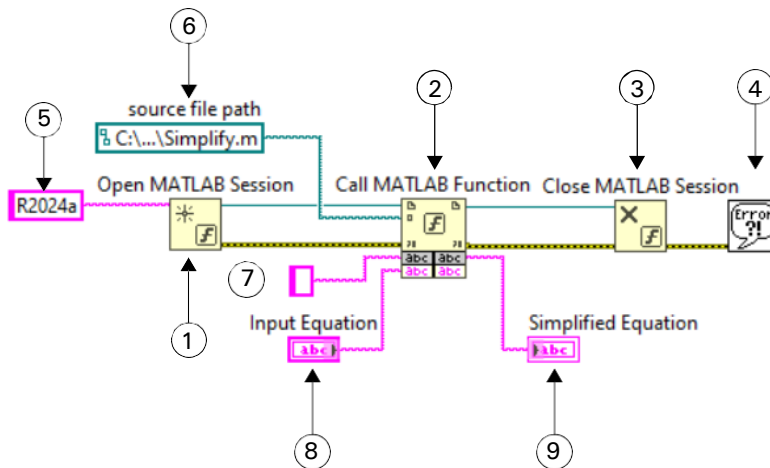
- ☐ Save the new function file and name it Simplify.m in the Exercises\Exercise 19-1 Simplifying Equations with MATLAB in NI LabVIEW destination

3. Create a new VI in the project.

- ☐ In the **Project Explorer** window, add a new VI.
- ☐ Press <Ctrl-S> to save the VI and name it Simplify Mathematical Equations.

4. Modify the block diagram as shown in the following figure.

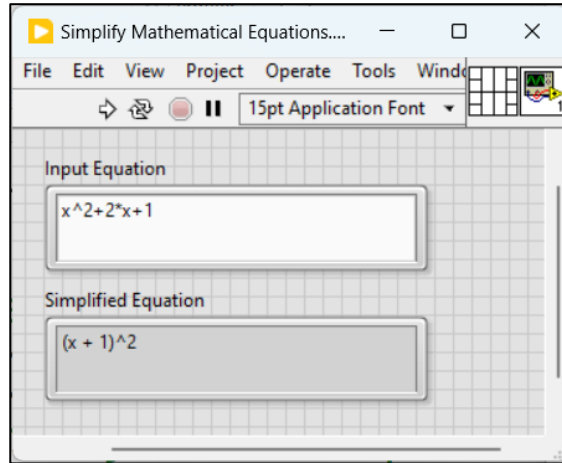
Figure 1. Simplify Mathematical Equations VI Block Diagram



- Open MATLAB Session** — Opens a MATLAB® session. If you have multiple MATLAB® versions installed, connect the release name terminal to a string constant and specify your MATLAB® release version (e.g., R2024a).
- Call MATLAB Function** — Use this subVI to call the `Simplify.m` function. Resize the Call MATLAB Function node to add one terminal.
 - ☐ Right-click on the source file path terminal and choose Create Constant.
 - ☐ Right-click on the constant and select **Browse for Path...** Navigate to your project directory and select the `Simplify.m` file.
- Close MATLAB Session** — Closes a MATLAB® session.
- Simple Error Handler VI** — Add the Simple Error Handler VI to indicate whether any errors occurred during the MATLAB function call.
- Release name constant** — specifies the release of MATLAB® in which the MATLAB session runs.
- Source file path constant** - specifies the absolute path to the source .m file.
- Empty String constant** — Add an Empty String Constant to specify the return type for the Call MATLAB Function node. Connect this to the Call MATLAB Function return type terminal.
- Input Equation string control** — Add a String Control to input the equation. Rename it Input Equation and connect it to the Call MATLAB Function input parameter terminal.
- Simplified Equation string indicator** — Add a String Indicator to display the result of the MATLAB function. Rename it Simplified Equation and connect it to the Call MATLAB Function return value terminal.

5. Modify the front panel shown in Figure 2.

Figure 2. Simplify Mathematical Equations VI Front Panel



1. Resize the String Control (**Input Equation**) and String Indicator (**Simplified Equation**) to ensure they are large enough to display lengthy equations clearly.
2. Rearrange the controls and indicators by placing the **Input Equation** (String Control) above the **Simplified Equation** (String Indicator) for a logical top-to-bottom flow.

6. Save the VI.

Test

Simplify symbolic equations using the Simplify.m function in MATLAB through NI LabVIEW. Observe the results and verify accuracy.

1. Run the VI.
 - ☐ Enter a symbolic equation in the Input Equation control (e.g., $x^2 + 2*x + 1$).
 - ☐ Click Run to execute the VI.
2. Observe Results.
 - ☐ Check the **Simplified Equation** indicator to see the simplified result (e.g., $(x + 1)^2$).
 - ☐ Verify that the output matches the expected simplification result.
3. Test with Additional Equations:
 - ☐ Repeat the process with different symbolic equations and write simplified equations below:
 - $\sin(x)^2 + \cos(x)^2$ _____
 - $x^3 + 3*x^2 + 3*x + 1$ _____
 - ☐ Observe how the function handles each case and confirm the accuracy of the results.

End of Exercise 1-1

1.3. NI LabVIEW and Python Integration

In this section, you will learn how to integrate Python scripts into NI LabVIEW.

Python has grown in popularity and expanded to offer hundreds of thousands of libraries for a wide spectrum of applications such as data analysis, machine learning, automation, and scientific computing. Incorporating Python into NI LabVIEW merges the flexible scripting capabilities of Python with the system design tools in NI LabVIEW.



Note This functionality was introduced in NI LabVIEW 2018 and is not available in previous versions.

Installing and Configuring Python Environment

Before you begin, use the following table to determine which version of Python is compatible with your NI LabVIEW version. Although unsupported versions might work with the NI LabVIEW Python functions, NI recommends only using supported versions of Python.

Before you begin, check the [LabVIEW and Python compatibility](#) to determine which version of Python is compatible with your NI LabVIEW version. Although unsupported versions might work with the NI LabVIEW Python functions, NI recommends only using supported versions of Python.



Note NI LabVIEW and Python must have the same bitness to ensure compatibility and proper functionality.

When you identify the compatible version of Python, you need to install Python to use the NI LabVIEW Python functions and to run the Python code. Also note that NI LabVIEW requires the Python shared libraries in the system path.

Follow steps for your operating system to add the Python shared libraries to the system path.

Windows: Download and install the Python Interpreter in a compatible version. You can either use the pure Python Interpreter installation, or a IDE which also installs the Python Interpreter. NI LabVIEW will then use registry entries to find the installed python3.x interpreters. Alternatively, use the Python path input of the Open Python Session.vi.

Linux: Use the package manager of your Linux Distribution to get the Python Interpreter in a compatible version. You can either use the pure Python Interpreter installation, or a IDE which also installs the Python Interpreter. NI LabVIEW search in /bin/python3.x, /usr/local/bin/python3.x and /usr/bin/python3.x for installed python3.x interpreters. Alternatively, use the Python path input of the Open Python Session.vi.

NI LabVIEW Python Palette

Python palette provides tools to open and close Python sessions and run Python code through the Python Node. These functions enable seamless integration of Python scripts into NI LabVIEW projects.

Function	Terminal	Description
Open Python Session		This subVI initializes a Python environment within NI LabVIEW. It establishes a connection between NI LabVIEW and Python, enabling communication between the two environments.
Python Node		The Python Node subVI serves as a bridge between NI LabVIEW and Python functionalities. Users can pass data between NI LabVIEW and Python environments, execute Python scripts, and call Python functions directly within NI LabVIEW using this node.

Close Python Session		Close Python Session subVI terminates the Python environment initiated by the Open Python Session subVI. It ensures proper cleanup and releases any resources associated with the Python session, promoting efficient memory management within the NI LabVIEW environment.
Open Virtual Env Session		Opens a Python session with a specific version of Python in a virtual environment or in an Anaconda. <i>Note: This function is available started NI LabVIEW 2023 Q1 version</i>
Python Object Refnum		The refnum output on Python node represents a Python class object, which can then be passed as an input to another Python Node function with the ability to act on the class object. <i>Note: This function is available started NI LabVIEW 2022 Q3 version</i>
Close Reference		Closes a refnum associated with an open VI, VI object, an open application instance, or a .Net, ActiveX or Python object.



Note Python Node is not supported on real-time or FPGA targets.



Tip You can resize the Python Node to add more terminals corresponding to the number of input arguments for your Python function.

Supported Data Types

Python node automatically converts NI LabVIEW data types to the corresponding data types in Python and vice versa. In the following table you can see the supported data types in NI LabVIEW and their corresponding data type conventions.

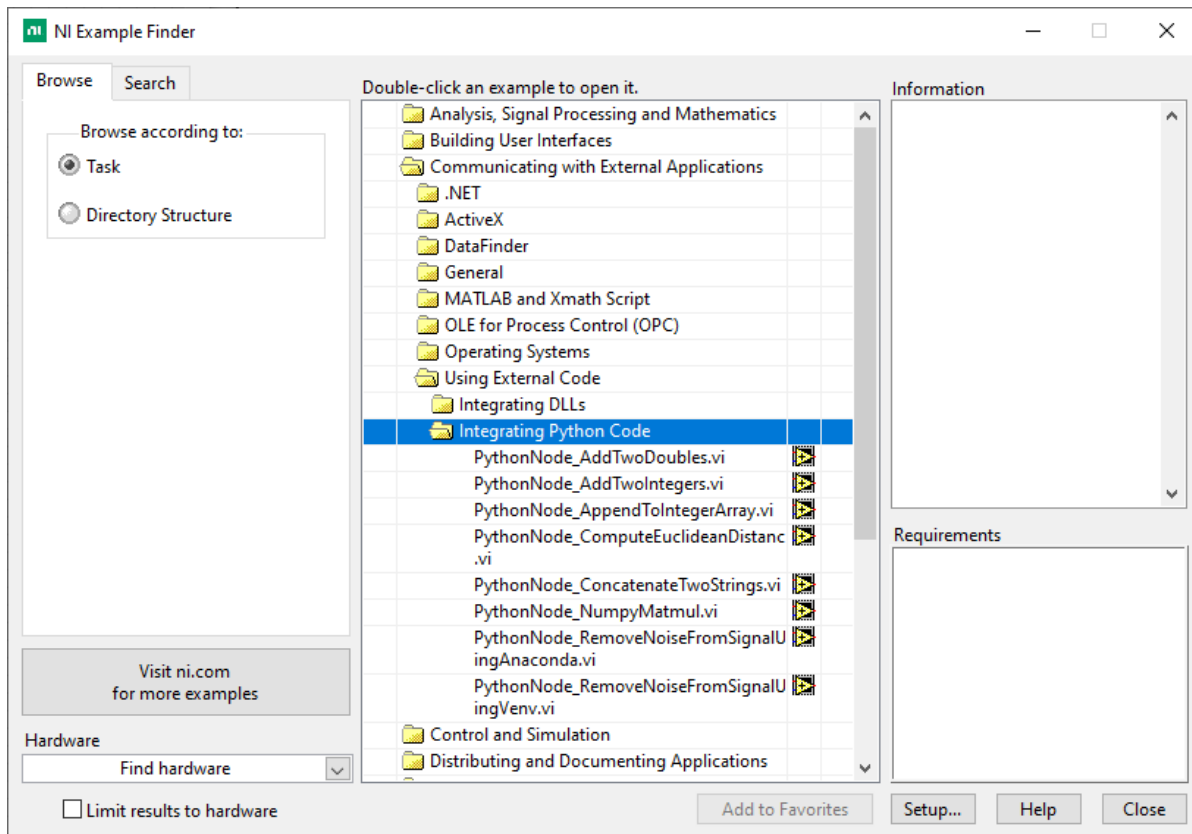
LabVIEW Data Type	Python Data Type
Integers	int
SGL, DBL	float
String	str
Boolean	bool
Array (including multi-dimensional arrays)	list or Numpy array
Cluster	Tuple or Namedtuple
Python Object Refnum	Object Reference

By default, the Python Node marshals arrays to lists. To marshal an array wired to **input parameter** to a NumPy array, right-click **input parameter** and select **Marshal to NumPy Array** from the shortcut menu.

By default, the Python Node marshals clusters to tuples. In Python, you can only reference items in a tuple by index and iterator. Python supports tuples, which allow you to reference items by name as well. If you want to reference named tuple items by name, you must marshal clusters to named tuples. To marshal a cluster wired to an input parameter to a named tuple, right-click **input parameter** and select **Marshal to Namedtuple** from the shortcut menu.

NI LabVIEW Python Functions Examples

There are multiple examples from NI LabVIEW that demonstrate how to set up your block diagram and call your Python code. Examples for using the Python Node are available in the NI LabVIEW Example Finder (found in Help >> Find Examples) under Communicating with External Applications >> Using External Code >> Integrating Python Code.



Activity 1-1: Explore Python and NI LabVIEW Integration Examples

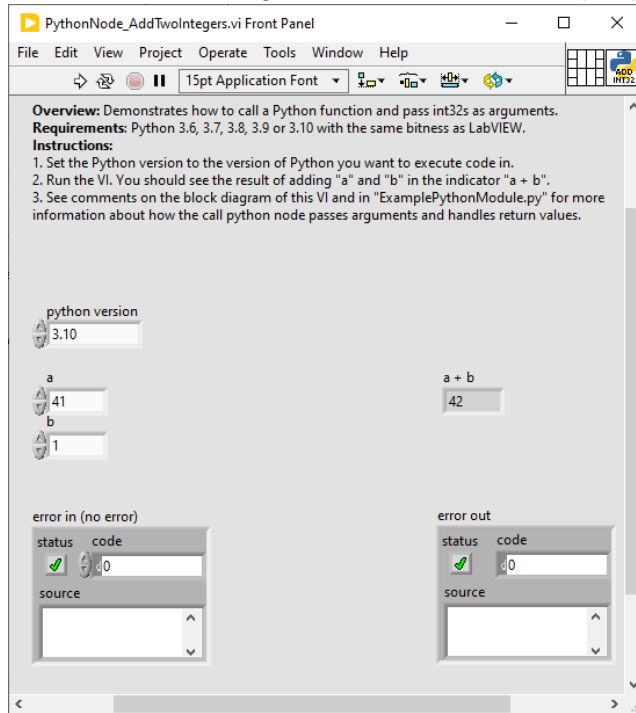
Goal

Investigate and learn how to call Python scripts into NI LabVIEW.

Instructions

1. Launch NI LabVIEW. Select Help»Find Examples to start the NI Example Finder.
2. From the Browse Tab choose Task option.
3. Under Communicating with External Applications » Using External Code » Integrating Python Code » Open PythonNode_AddTwoIntegers.vi
4. Set the Python version to the version of Python you want to execute code in.

5. Run the VI. You should see the result of adding "a" and "b" in the indicator "a + b".
6. Read comments on the block diagram of this VI and in "ExamplePythonModule.py" for more



information about how the call python node passes arguments and handles return values. Also use the Context Help and investigate the VI's in the block diagram.

1.4 Exercise 1-2: Calling Python Objects from NI LabVIEW

Goal

This exercise will demonstrate how to use Python object in a NI LabVIEW environment.

Description

During this exercise, you will implement a Python class with methods and use the NI LabVIEW Python palette to initialize the object and call its methods. You will learn how to pass data between NI LabVIEW and Python, enabling you to leverage Python's functionality within your NI LabVIEW projects.



Note To complete this exercise, ensure that NI LabVIEW 2022 Q3 or a later version is installed, and that a compatible version of Python is also installed with its path added to the system environment variables.

1. Create a blank project and save it as Exercises\Exercise 19-2 Calling Python Objects from LabVIEW \ Calling Python Objects from LabVIEW.lvproj.
2. Open any Python IDE and create a new empty script.
 - ☐ Prepare a Python script with object implemented. For easier understanding the following Python script will be used throughout this exercise.

```
# Define a class for incrementing a number by a fixed value
class IncrementClass:
    def __init__(self, i):
        # Initialize the object with the increment value 'i'
        self.i = i

    def increment(self, a):
        # Add the stored increment value to the input 'a' and return the result
        res = a + self.i
        return res

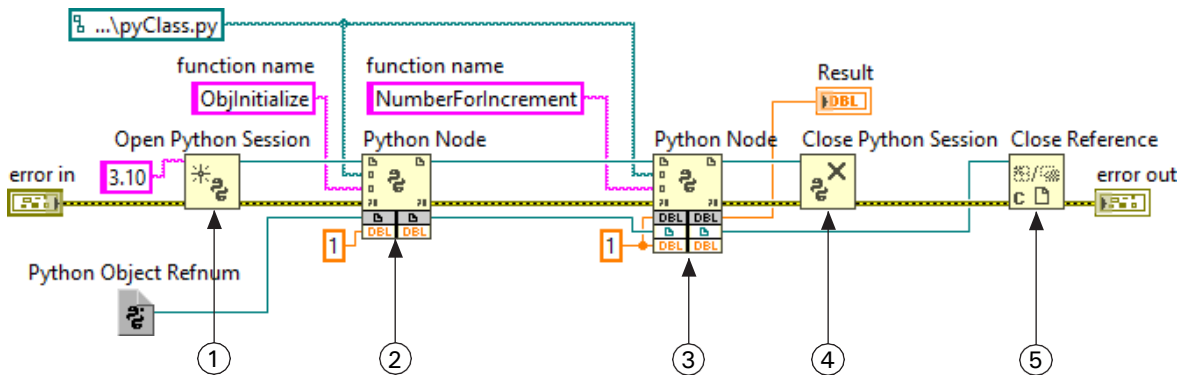
# Function to use an IncrementClass object to increment a number
def NumberForIncrement(obj, a):
    # Call the increment method of the object with input 'a'
    s = obj.increment(a)
    return s

# Function to initialize an IncrementClass object with a given value
def ObjInitialize(i):
    # Create and return an instance of IncrementClass with increment value 'i'
    obj = IncrementClass(i)
    return obj
```

- ☐ Save the new script file and name it pyClass.py in the Exercises\Exercise 19-2 Calling Python Objects from LabVIEW destination.
3. Create a new VI in the project.
 - ☐ In the **Project Explorer** window, add a new VI.
 - ☐ Press <Ctrl-S> to save the VI and name it Calling Python Objects from LabVIEW.

4. Modify the block diagram as shown in the following figure.

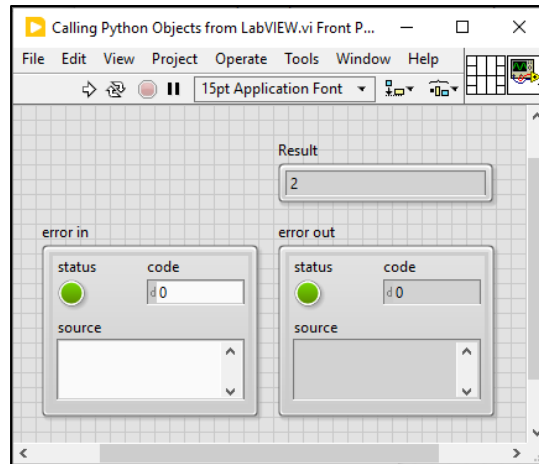
Figure 3. Calling Python Objects from LabVIEW VI Block Diagram



- 1 **Open Python Session** — Opens a Python session. Connect the Python version terminal to a string constant and specify your Python version (e.g., 3.10).
- 2 **Python Node 1** — Use this subVI to call the `ObjInitialize` functions from `pyClass.py` file. This function creates and returns an object of `IncrementClass` with an increment value.
 - ☐ Right-click on the module path terminal and choose **Create » Constant**.
 - ☐ Right-click on the constant and select **Browse for Path...** Navigate to your project directory and select the `pyClass.py` file.
 - ☐ Right-click on the function name terminal and choose **Create » Constant**. Write `ObjInitialize`.
 - ☐ Create a **Python Object Refnum Constant** and connect it to the Python Node return type terminal.
 - ☐ Resize the Python Node to add one input terminal.
 - ☐ Connect **DBL Numeric Constant** to the Python Node input parameter terminal. Set it to 1 to increment the value by 1.
- 3 **Python Node 2** — Use this subVI to call the `NumberForIncrement` functions from `pyClass.py` file. This function calls the `increment` method of the object.
 - ☐ Connect the module path terminal to the module path constant.
 - ☐ Right-click on the function name terminal and choose **Create » Constant**. Write `NumberForIncrement`.
 - ☐ Resize the Python Node to two input terminals.
 - ☐ Connect the first Python Node return value to the second Python Node's first input parameter terminal.
 - ☐ Connect a **DBL Numeric Constant** to both the Python Node return type and input parameter terminals.
 - ☐ Right-click on the return value terminal and choose **Create » Indicator**. Rename it to `Result`.
- 4 **Close Python Session** — Closes the Python session.
- 5 **Close Reference** — Closes the Python object refnum.

5. Modify the front panel shown in Figure 4.

Figure 4. Calling Python Objects from LabVIEW VI Front Panel



1. Resize the Numeric Indicator (Result) to ensure they are large enough to display the returned values.
2. Rearrange the controls and indicators.

6. Save the VI.

Test

Observe the results and verify accuracy.

1. Click Run to execute the VI.
 2. Observe Results.
- ☐ Check the **Result** indicator to see the returned result.
 - ☐ Verify that the output matches the expected result.

Challenge

Objective: Modify the code, to allow users to enter the Increment Value and Input from the Front Panel.

End of Exercise 1-2

1.5 Calling Shared Libraries in NI LabVIEW

In this section, you will learn how to call shared libraries in NI LabVIEW.

Overview of Shared Libraries

Shared Library is a software module containing executable code and data that can be called by applications or other shared libraries. Shared libraries have several features and benefits:

- Functions and data in a shared library are loaded and linked at run time
- Shared libraries can be written in a variety of languages.
- Shared libraries allow you to call existing code in another language.
- Most modern development environments provide support for creating shared libraries.
- Shared libraries expose functions and data through a standardized interface.
- Most types of shared library definitions are similar to function definitions in the C programming language.

Shared libraries are often called by different names depending upon the platform where they are used.

- Windows = Dynamic Link Libraries (DLLs)
- MacOS = Frameworks
- Unix = Shared Libraries



Note 64-bit DLL cannot be loaded into a 32-bit version of NI LabVIEW, and vice versa.

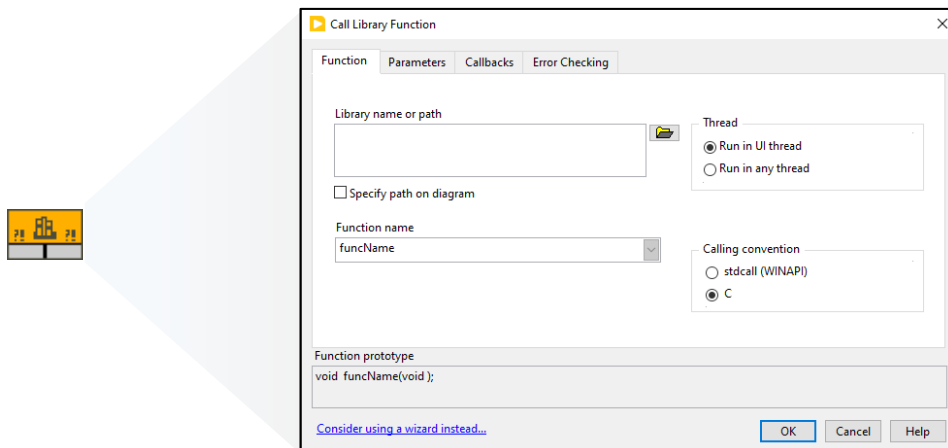
Calling a DLL from NI LabVIEW

There are two methods to configure the Call Library Function Node (CLFN) to use shared libraries in NI LabVIEW.

1. Configuring the CLFN Manually
 - This method is an alternative when using the wizard is not possible.
 - Configure the CLFN manually if you do not have a header file or are using unsupported data types.
2. Using the Import Shared Library Wizard
 - This is the recommended way to configure the CLFN. It allows NI LabVIEW to generate code to call the shared library.
 - Use this method if you have a header file to configure the CLFN automatically and if you are using supported data types.

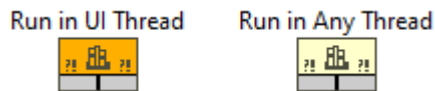
Configuring Call Library Function Node (CLFN) Manually

The Call Library Function Node is a powerful feature in NI LabVIEW that allows developers to access and execute functions from external shared libraries, such as dynamic link libraries (DLLs). This capability significantly extends NI LabVIEW's functionality by enabling the use of pre-existing external code. To use the node effectively, understanding its configuration and requirements is essential.

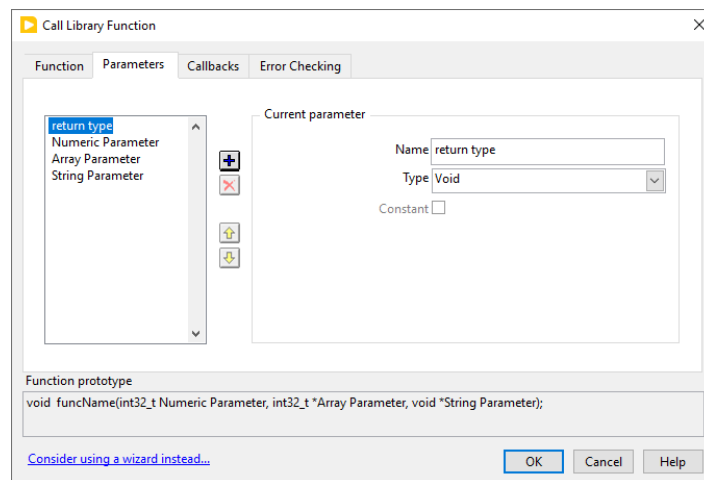


When configuring the Call Library Function Node, several key settings must be specified:

1. **Library Name or Path:** This is the file path to the external library (e.g., a .dll file) containing the desired function.
2. **Function Name:** Specifies the name of the function you are calling. Note When you reference a library, the Function name control is populated with the function names found in that library. (macOS) If the Function name control is empty, install the nm utility to see the list of function names. The nm utility is installed with Xcode, the standard development environment for macOS. You can download Xcode for free from the Mac App Store.
3. **Threading Options:** NI LabVIEW provides two threading options for executing the external function:
 - **Run in UI Thread:** Specifies the Call Library Function Node execution must switch from the current thread to the user interface thread. Execution in the UI thread of a NI LabVIEW-built shared library that is dynamically loaded using Specify path on diagram causes NI LabVIEW to hang.
 - **Run in Any Thread:** Specifies that the Call Library Function Node execution continues on the current thread executing the VI. You must make sure that multiple threads can call the function simultaneously. Note All calls to NI LabVIEW-built shared libraries should specify Run in any thread. If you configure the Call Library Function Node using NI LabVIEW-built shared libraries and specify Run in UI thread, NI LabVIEW might hang and require you to restart.

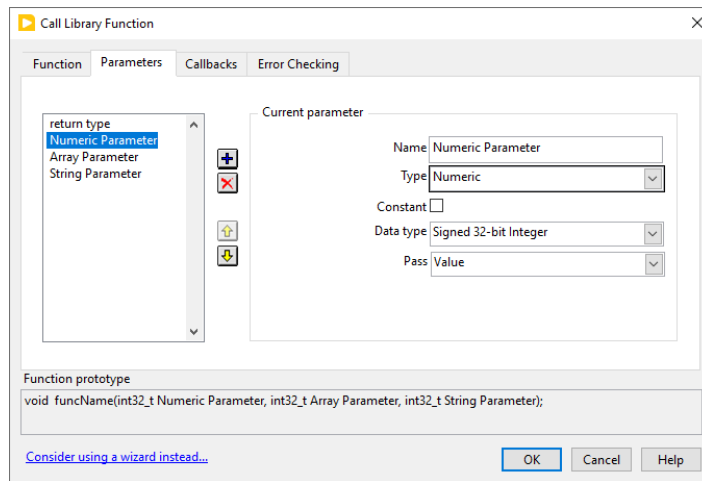


4. **Calling Convention:** The calling convention determines how function arguments are passed and how the stack is managed during function calls. NI LabVIEW supports the following calling conventions:
 - **stdcall (WINAPI):** Common in Windows APIs, this convention places responsibility for stack cleanup on the called function. It is the default for most DLL functions.
 - **C declaration:** Used in many C libraries, this convention requires the caller to clean up the stack, allowing functions with variable-length argument lists.
5. **Parameter Setup:** Each parameter used by the function must be configured. Lists the current parameters for the Call Library Function Node. You can add a new arguments/parameter and configure each argument data type. **Return type** is a required parameter that you cannot delete. Ensure all parameter configurations match the specifications of the external library to avoid runtime errors.



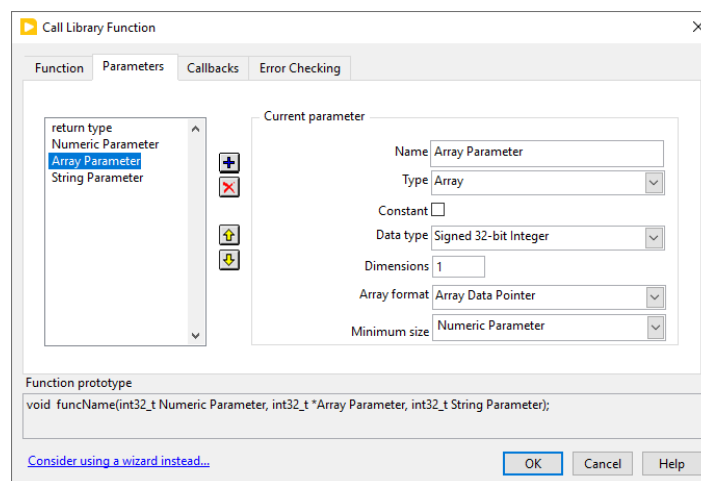
Common parameter types are the following:

- **Numeric** – Specify the numeric representation. Determine whether to pass by value or by reference.



- **Array** – Select the data type of the array. Select the number of dimensions for the array. Determine how to pass the Array (array format). Create another parameter to specify the array size if the Array format is Array Data Pointer.

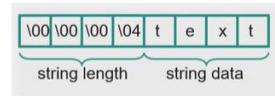
Array Formats		
Format	Type	Details
Array Data Pointer	* Array	Pointer to the first element of the array. Shared library cannot change the array's location in memory.
Array Handle	** Array	Pointer to a pointer to the array size and array data. Shared library can resize the array.
Array Handle Pointer	*** Array	Pointer to a pointer to a pointer of the array size and array data. - Shared library can resize the array. - Shared library can delete the array or allocate a new array.



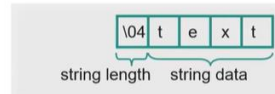
- **String**- Used to specify the String format. The available string formats are as follows:
 - **String Handle** —a pointer to a pointer to four bytes for length information, followed by string data.
 - **String Handle Pointer** —a pointer to an array of string handles.
 - **C String Pointer** —a string followed by a null character.

- **Pascal String Pointer** — a string preceded by a length byte.

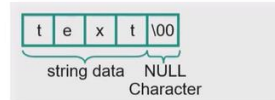
LabVIEW String Handles



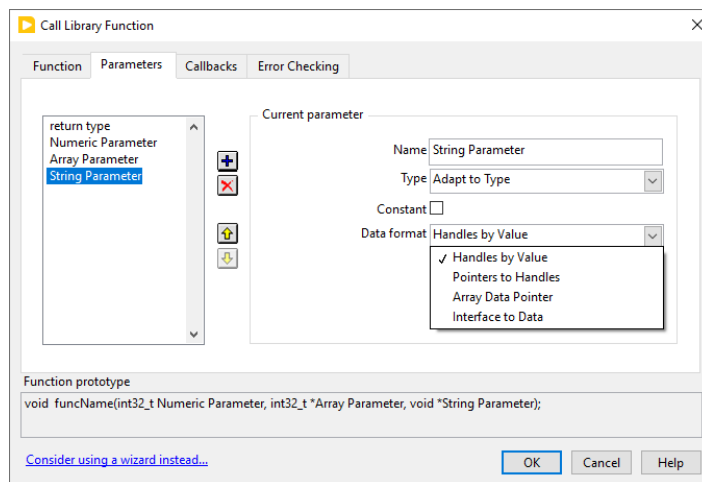
Pascal String Format



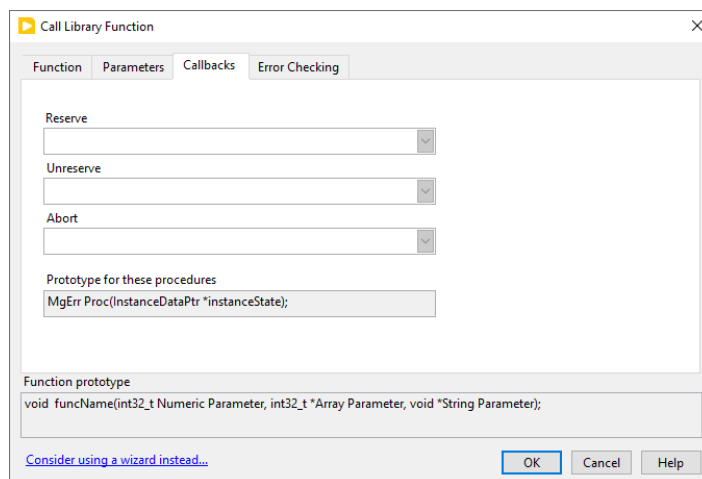
C String Format



- **Adapt to Type**- Used to pass arbitrary NI LabVIEW data types to DLLs. Scalars are passed by reference, arrays and strings are passed according to their data format, and clusters are passed by reference. The Call Library Function Node does not enforce the type of items wired to it.



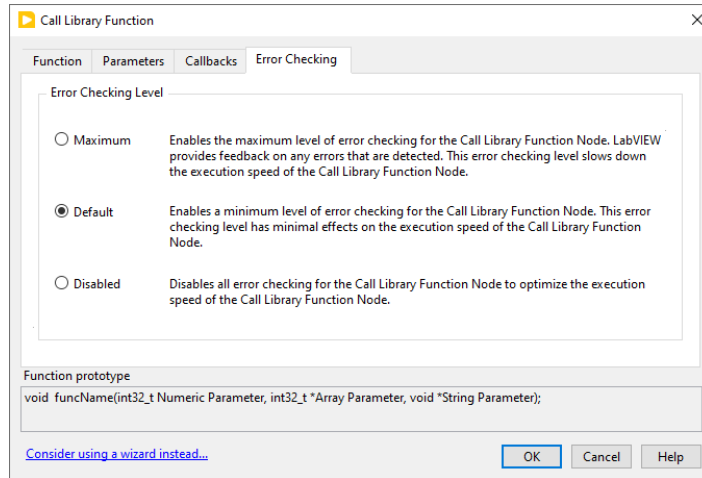
6. **Callbacks**- Callbacks allow you to automatically call functions from the DLL when certain events occur in the program. Callback are used to clean up resources allocated by the DLL in the event of an aborted VI.



7. **Error checking** - You can specify the error checking level for the Call Library Function Node to control how errors are handled during execution:
 - **Maximum**: Enables thorough error checking, including warnings if the called function writes beyond allocated space for strings or arrays. Use this option only during debugging, as it reduces execution

speed and increases memory usage.

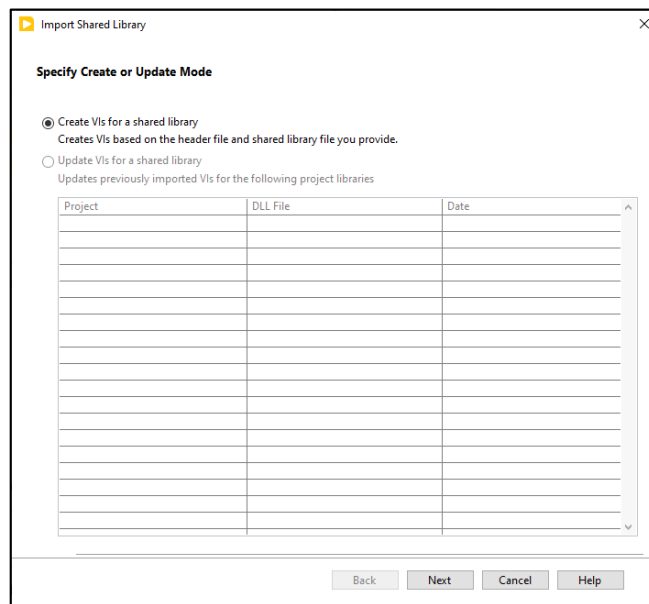
- **Default:** Provides standard error checking, helping NI LabVIEW recover from unhandled exceptions and ensuring the selected calling convention matches the function's requirements.
- **Disabled:** Turns off error checking to improve execution speed. Use cautiously, as unhandled errors may cause NI LabVIEW to shut down irregularly.



Note The detailed descriptions of the Configuring the Call Library Function Node you can find here: [Configuring CLFN](#).

Using the Import Shared Library Wizard

The Import Shared Library Wizard in NI LabVIEW simplifies the process of interfacing with shared libraries by automating the generation of wrapper VIs for the library functions. Import Shared Library Wizard generates Call Library Function Nodes and creates a library of subVIs from shared libraries. To start the wizard, open NI LabVIEW, then select **Tools > > Import > > Shared Library**.



Note Import Shared Library Wizard requires the header file of the library.

The wizard has the following advantages:

- Reduces time to create and edit VI wrappers.
 - Automates process using a configuration-based wizard.
 - Quickly configures calls to multiple functions.
 - Stores configuration information to refine the generated API.
- Converts shared library data types to NI LabVIEW data types.
 - Converts pointers (strings, arrays, etc.) to values.
 - Converts structs to clusters.

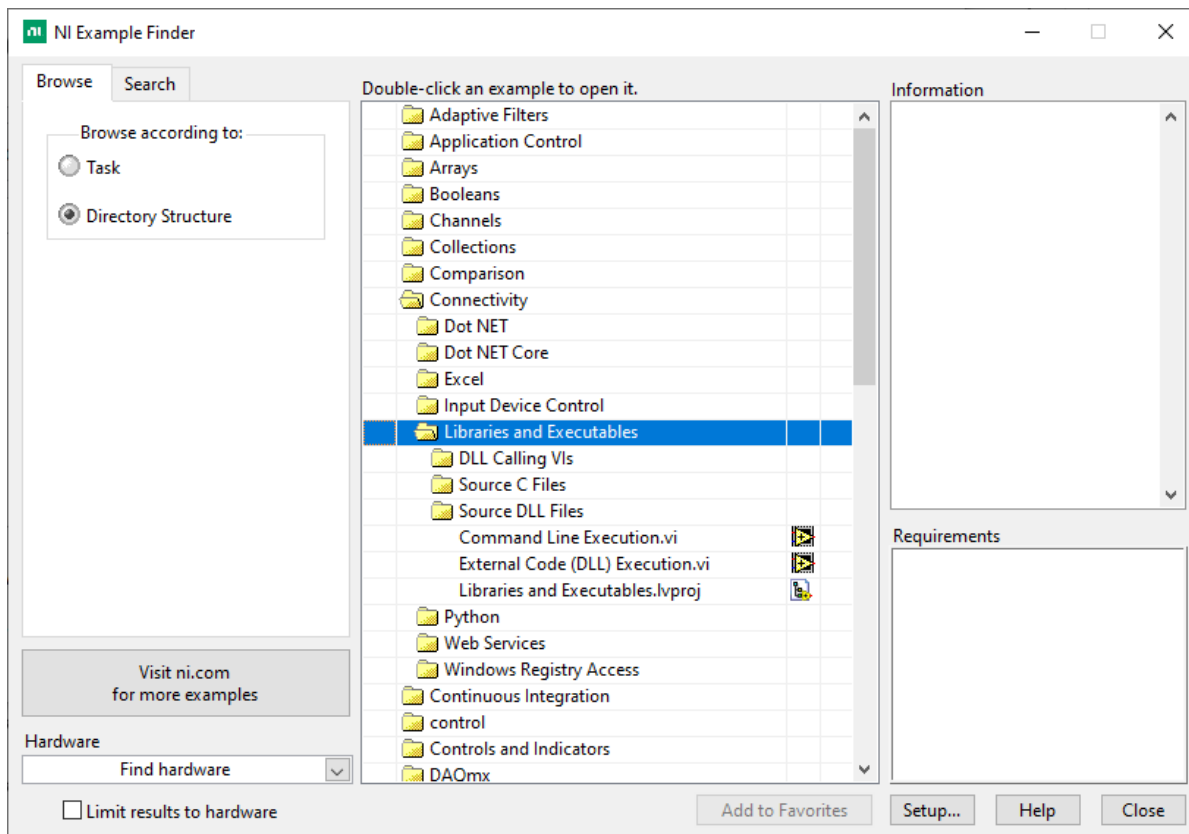
Limitations of using the Import Shared Libraries Wizard are the following:

- Requires knowledge of C prototypes.
 - Greater familiarity allows better optimization of NI LabVIEW wrapper VIs.
- Must have a header file for the shared library.
 - Custom header files to simplify parsing complex header files might be required.
- Some complex parameter types are not supported.
 - Examples: array of structures, structures as return values.
 - To work around these limitations, you may need to create a custom control for the parameter and wire an instance of that type to the Call Library Function Node.

NI LabVIEW Integrating DLLs Examples

There are multiple examples from NI LabVIEW that demonstrate how to set up your block diagram and call your DLL functions.

Examples for calling DLLs are available in the NI LabVIEW Example Finder (found in Help > Find Examples) under Connectivity > Libraries and Executables.



1.6.Exercise 1-3: Building and Importing a C++ Dynamic Library in NI LabVIEW

Goal

This exercise will demonstrate how to use build and import DLLs in NI LabVIEW environment.

Description

During this exercise, you will create a dynamic library in C++ and learn how to import it into NI LabVIEW using the Import Shared Library Wizard. You will explore how to define functions in a C++ dynamic link library (DLL) and call them from NI LabVIEW, passing data between the two environments.

1. Open any C/C++ IDE and create a new project.
2. Create the Header File.
 - ☐ Create a header file that contains the declaration of the functions that are to be implemented. The `mymath.h` header file defines the function prototypes for basic mathematical operations: addition, subtraction, multiplication, and division.

```
// mymath.h
// header guards
#pragma once
#ifndef MYMATH_H
#define MYMATH_H

// declaring functions
int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
int divide(int a, int b);
```

- ☐ The following header file will be used throughout this exercise.
- ☐ Name the header file as `mymath.h`.



Tip Function implementations can be added directly to the header file, it is generally best practice to keep them in a separate `.cpp` file to maintain a clean and modular code structure.

3. Create the Implementation File for the Header File
 - ☐ After declaring the header file, create an implementation file that will contain the actual code for the functions declared in the header file. This file will consist of code for the function definitions which will be executed when the functions are called.
 - ☐ The following implementation file will be used throughout this exercise.

```
// including header
#include "mymath.h"

// addition implementation
int add(int a, int b) { return a + b; }

// subtraction implementation
int subtract(int a, int b) { return a - b; }

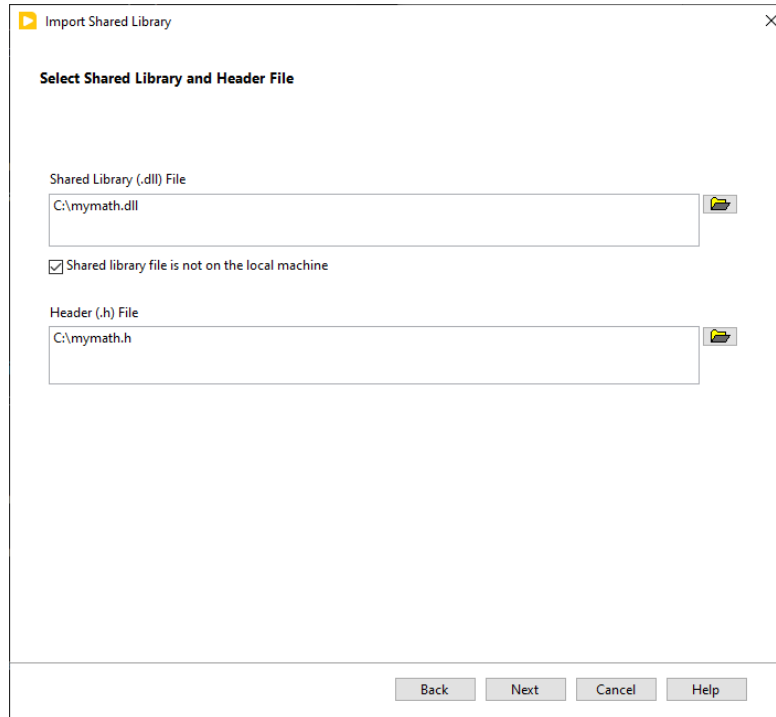
// multiply implementation
int multiply(int a, int b) { return a * b; }

// divide implementation
int divide(int a, int b) { return a / b; }
```

- ☐ Name the implementation file as `mathlib.cpp`.
4. Compile the source code into a dynamic library. Note that the dynamic library extension depends on your operating system:
 - ☐ **Windows:** `.dll`
 - ☐ **Linux:** `.so`
 - ☐ **macOS:** `.dylib`
5. After compiling, you will have a dynamic library file. Name the output file `mymath.dll` (or the equivalent extension for your system).
6. Launch NI LabVIEW and navigate to **Tools >> Import... >> Shared Library (.dll)** to launch the Import Shared Library Wizard.
7. Select **Create VIs for a shared library** and then **Next**.

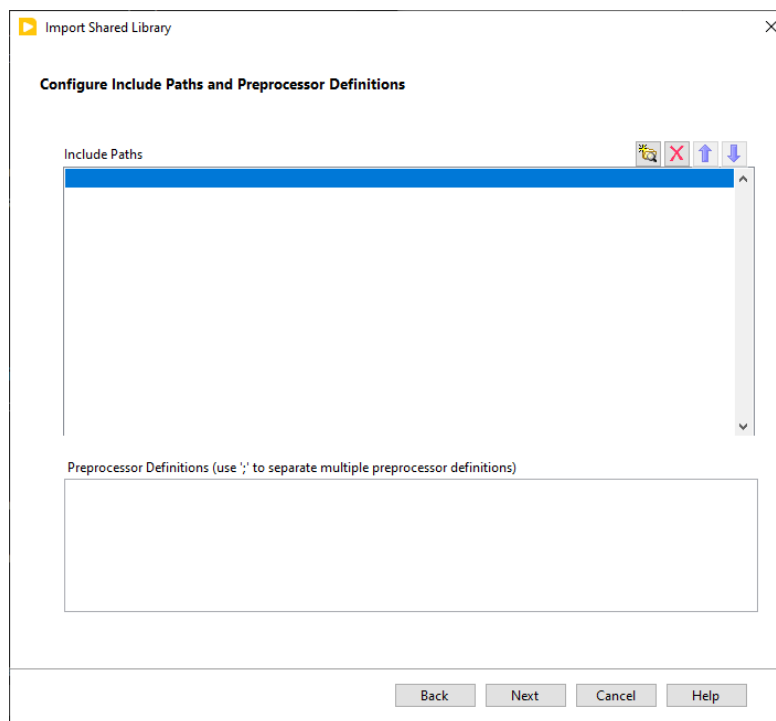
[illegible]

8. Input the file paths for the **Shared Library (.dll)** File and **Header (.h)** File. Ensure the "Shared library file is not on the local machine" option is checked. Then, click **Next**.



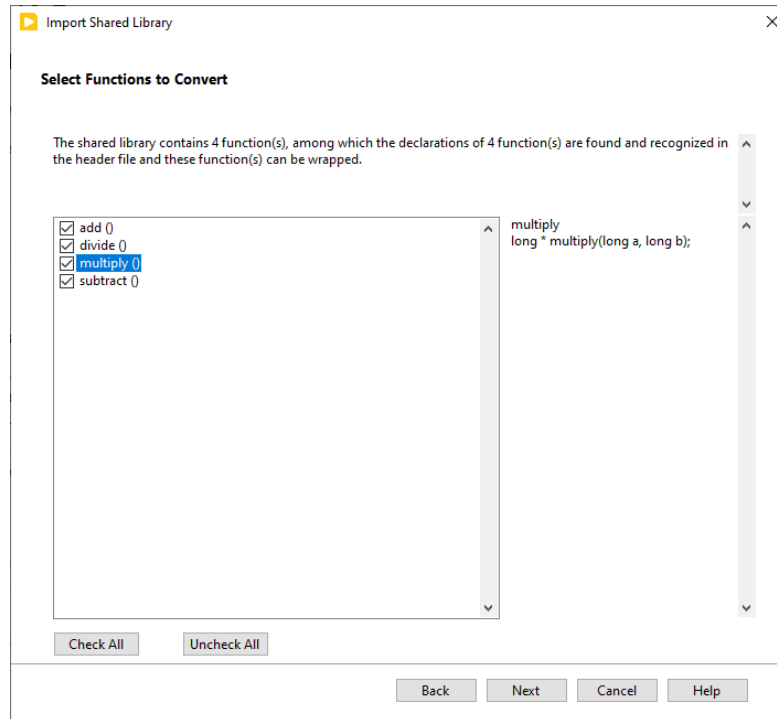
The screenshot shows the 'Import Shared Library' dialog box with the 'Select Shared Library and Header File' tab selected. The 'Shared Library (.dll) File' text box contains 'C:\mymath.dll' and the 'Header (.h) File' text box contains 'C:\mymath.h'. The checkbox 'Shared library file is not on the local machine' is checked. At the bottom are buttons for 'Back', 'Next', 'Cancel', and 'Help'.

9. In this section, you can configure the Input Paths and Preprocessor Definitions. For this exercise, we will leave the settings as they are and click **Next**.

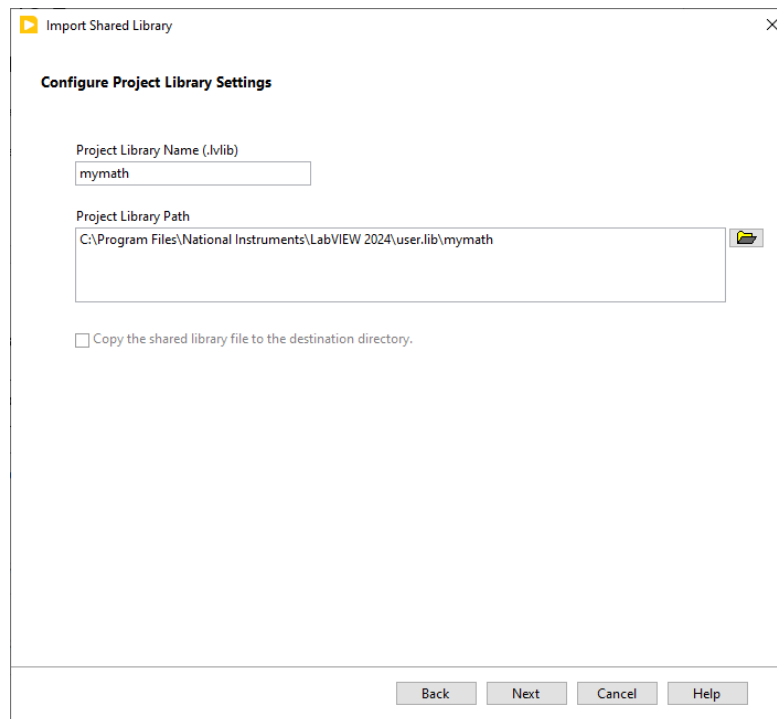


The screenshot shows the 'Import Shared Library' dialog box with the 'Configure Include Paths and Preprocessor Definitions' tab selected. The 'Include Paths' section has a list box with a blue header bar and icons for adding, removing, and moving items. The 'Preprocessor Definitions (use ';' to separate multiple preprocessor definitions)' section has an empty text box. At the bottom are buttons for 'Back', 'Next', 'Cancel', and 'Help'.

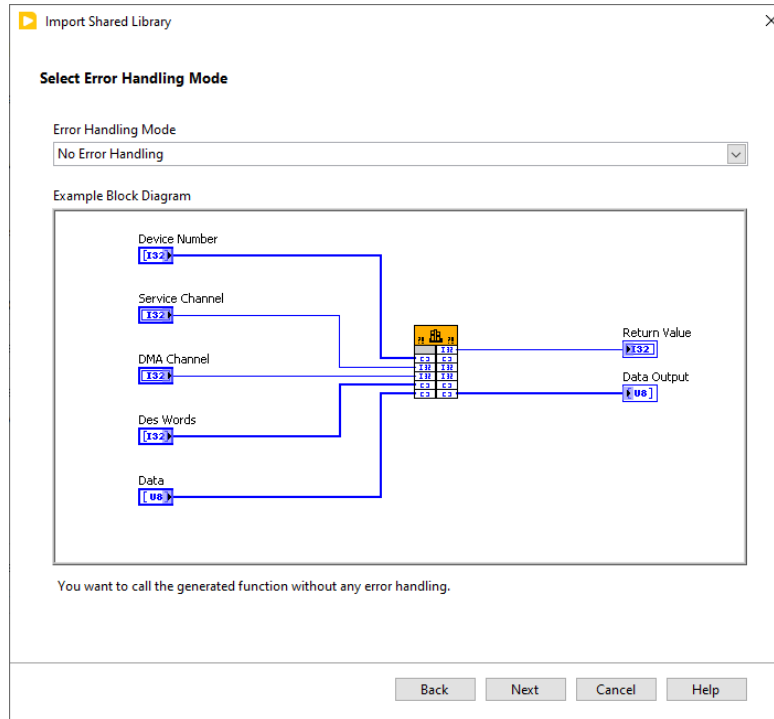
10. Check all functions to include them into NI LabVIEW Shared Library and then click **Next**.



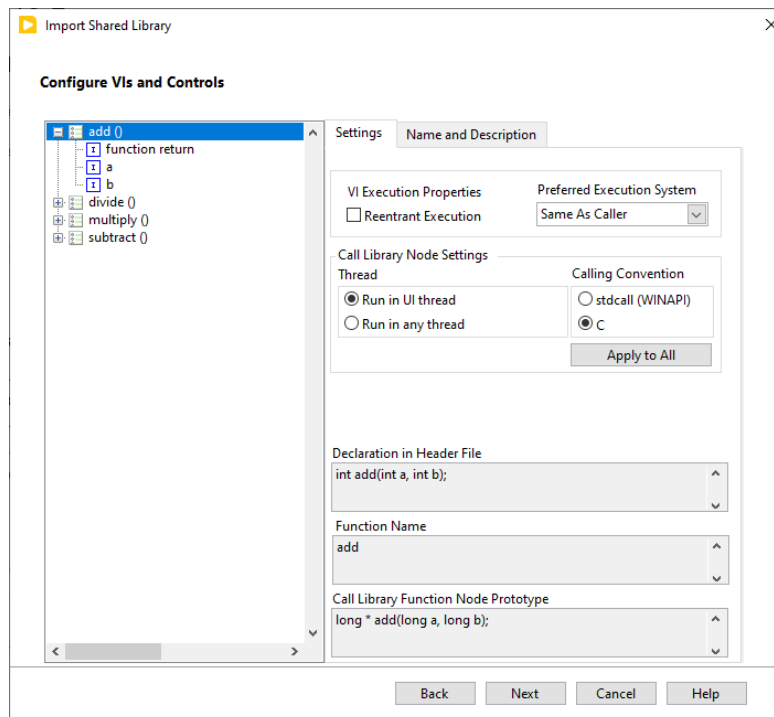
11. Specify the name of the NI LabVIEW project library file as `mymath`. The wizard will create this library to contain the VIs that wrap the imported functions. Set the directory where the wizard will save the generated files. By default, the files are saved in the `LabVIEW/user.lib/mymath` directory of the current LabVIEW version. You can enter a custom path or click the Browse button to select a directory. Once done, click **Next**.



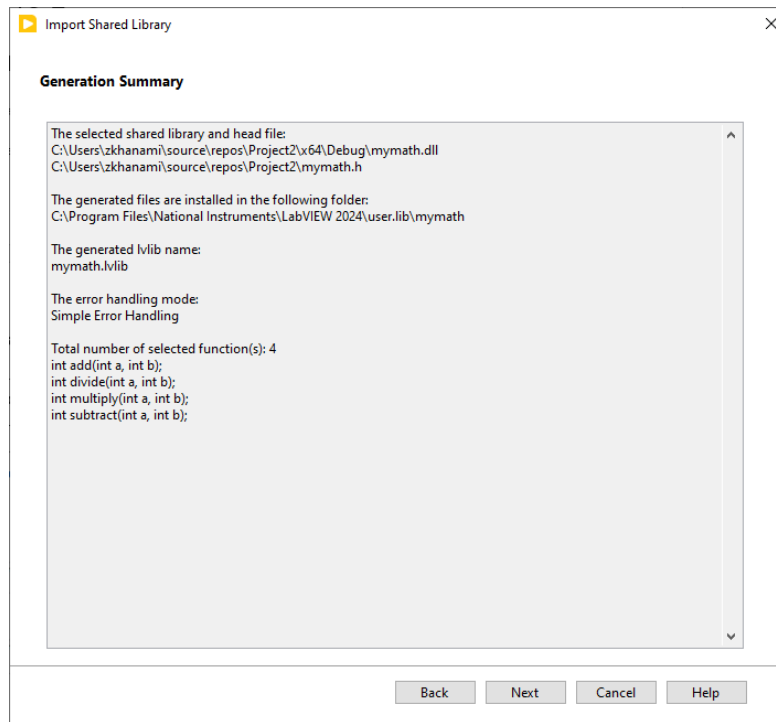
12. From the **Error Handling Mode** dropdown, select **Simple Error Handling** to enable basic error handling for the generated VIs. Once selected, click **Next**.



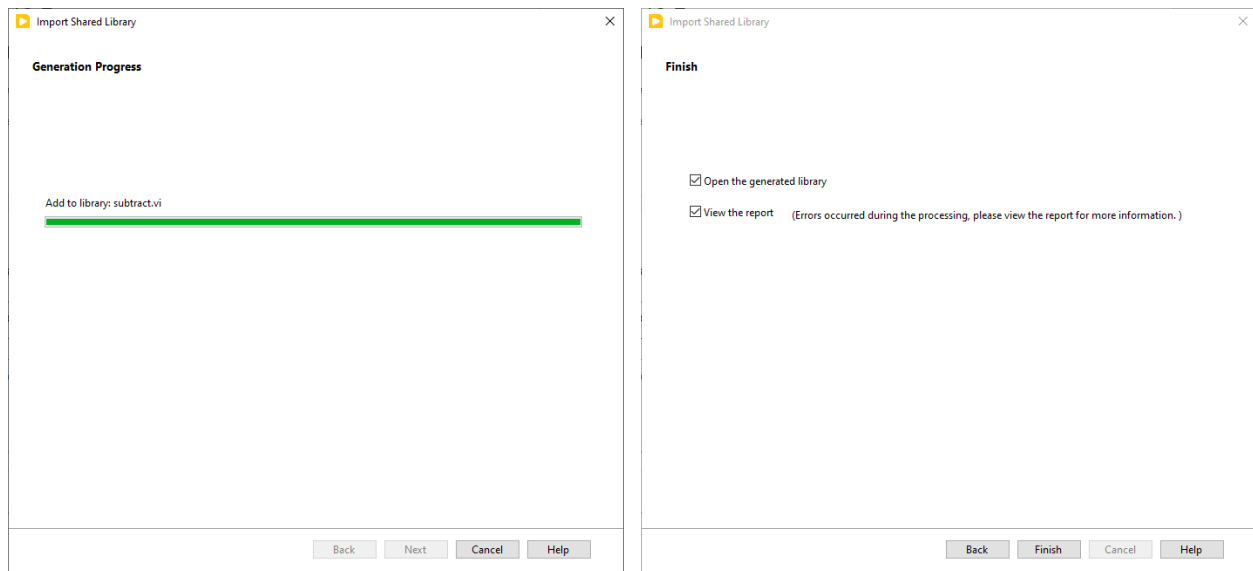
13. In this section, you can reconfigure the Vis and Controls, Call Library Node settings, calling conventions, and function declarations. For this exercise, leave the settings as they are and click **Next**.



14. In this section review the Generation Summary and then click **Next**.



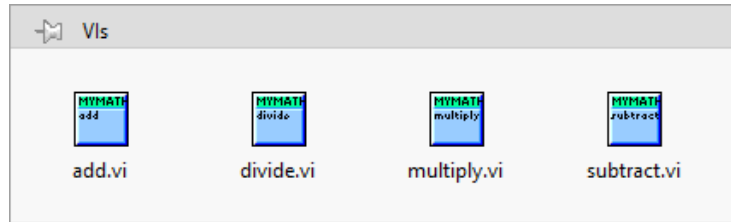
15. Wait for the generation process to complete. Once finished, click the **Finish** button.



16. To test the shared library create a blank project and save it as **Exercises\Exercise 19-3 Building and Importing a DLL in NI LabVIEW\ Import DLL Functions.lvproj**.

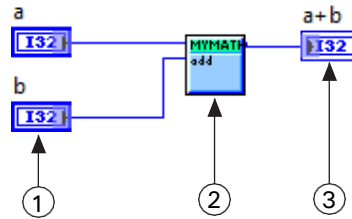
- ☐ In the **Project Explorer** window, add a new VI.
- ☐ Press **<Ctrl-S>** to save the VI and name it **Calling DLL**.

17. Check the generated library. If the previous steps were followed correctly you should find all four functions listed as Vis (right-click on the Block Diagram, go to **Functions >> User Libraries >> mymath >> VIs**).



18. Modify the block diagram as shown in the following figure.

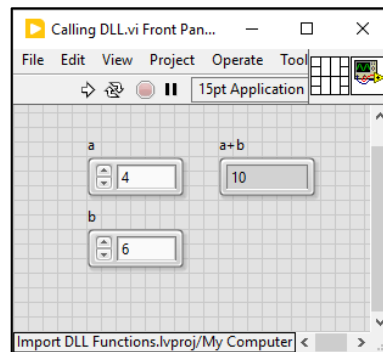
Figure 5. Calling DLL VI Block Diagram



- 1 **Numeric Controls** — These controls allow the user to input numerical values. The values entered here are passed to the relevant VIs for processing.
- 2 **MYMATH add VI** — This subVI calls the addition function from the `mymath` library, passing input values and returning the result. It performs the addition operation defined in the C++ dynamic library.
- 3 **Numeric Indicator** — This indicator displays the result of the addition operation

19. Modify the front panel shown in Figure 5.

Figure 6. Calling DLL VI Front Panel



20. Save the VI.



Tip In order to remove the list of imported libraries from this tool you will need to navigate to your `My Documents\LabVIEW Data\Shared Library` directory and delete two XML files. The files to be removed are `HeaderParserResult.xml` and `yourfunctions.dll.xml`. This will clear the list as well as remove the traces of the import data. If you do not wish to remove all your imported functions, you can modify the `HeaderParserResult.xml` file to not include the libraries you no longer want on the list.

Test

Observe the results and verify accuracy.

1. Click **Run** to execute the VI.

2. Observe the results:

- ☐ Check the **a + b** indicator to see the returned result.
- ☐ Verify that the output matches the expected result.

3. Add the other three VIs (subtract, multiply, and divide).

4. For each, observe the corresponding indicators and verify that the outputs match the expected results.

End of Exercise 1-3



Activity 1-2: Lesson Review

1. When using the Call MATLAB function, which are the required inputs?
 - a. Session in
 - b. Error in
 - c. Function name
 - d. Source file path
2. If a MATLAB function returns multiple values, how should the Call MATLAB function be configured?
 - a. Wire several return outputs for each returned value.
 - b. Wire a cluster with elements that correspond to the return values.
 - c. Wire an array with elements corresponding to the return values.
 - d. Modify the MATLAB function to return a single value.
3. What are some of the requirements to call a Python script from LabVIEW?
 - a. Add the directory for your Python version to the system path
 - b. Run Python in a virtual environment
 - c. Ensure that the bitness of Python corresponds to the bitness of LabVIEW
 - d. Make sure the Python script and dependencies are in the same folder
4. What is the required input to open a Python session from LabVIEW using the Open Python Session function?
 - a. Python path
 - b. Python version
 - c. Error input
 - d. Reference input
5. Which of the following do you need to know to call a function in a shared library?
 - a. Data type returned by the function
 - b. Calling convention
 - c. Development environment that created the shared library
 - d. Order of parameters to be sent to the function
6. Which of the following files does the Import Shared Library Wizard require to generate wrappers VIs for a library?
 - a. Wrapper VI template
 - b. Header file
 - c. Shared library
 - d. .lib file
7. What are some advantages of using the Import Shared Library Wizard?
 - a. Converts shared library data types to LabVIEW data types
 - b. Can be used with unsupported data types
 - c. Reduces time to create and edit VI wrappers
 - d. Does not need to provide a header file



Activity 1-2: Lesson Review - Answers

1. When using the Call MATLAB function, which are the required inputs?
 - a. **Session in**
 - b. Error in
 - c. Function name
 - d. **Source file path**
2. If a MATLAB function returns multiple values, how should the Call MATLAB function be configured?
 - a. Wire several return outputs for each returned value.
 - b. **Wire a cluster with elements that correspond to the return values.**
 - c. Wire an array with elements corresponding to the return values.
 - d. Modify the MATLAB function to return a single value.
3. What are some of the requirements to call a Python script from LabVIEW?
 - a. **Add the directory for your Python version to the system path**
 - b. Run Python in a virtual environment
 - c. **Ensure that the bitness of Python corresponds to the bitness of LabVIEW**
 - d. Make sure the Python script and dependencies are in the same folder
4. What is the required input to open a Python session from LabVIEW using the Open Python Session function?
 - a. Python path
 - b. **Python version**
 - c. Error input
 - d. Reference input
5. Which of the following do you need to know to call a function in a shared library?
 - a. **Data type returned by the function**
 - b. **Calling convention**
 - c. Development environment that created the shared library
 - d. **Order of parameters to be sent to the function**
6. Which of the following files does the Import Shared Library Wizard require to generate wrappers VIs for a library?
 - a. Wrapper VI template
 - b. **Header file**
 - c. **Shared library**
 - d. .lib file
7. What are some advantages of using the Import Shared Library Wizard?
 - a. Converts shared library data types to LabVIEW data types
 - b. Can be used with unsupported data types
 - c. **Reduces time to create and edit VI wrappers**
 - d. Does not need to provide a header file

2. NI LabVIEW and Third-Party Hardware Integration

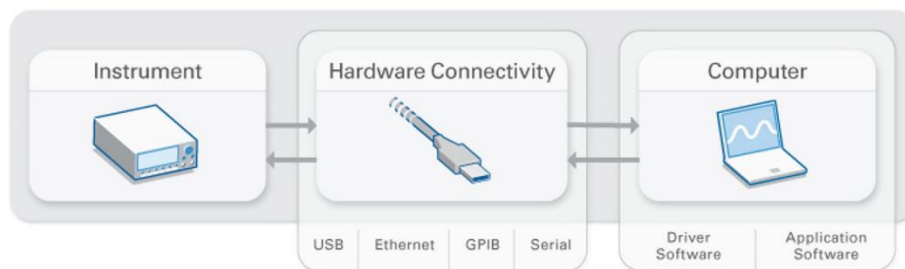
2.1. Automating Non-NI Instruments: NI-VISA

In this lesson you will learn how to use NI-VISA in NI LabVIEW to control and communicate with non-NI instruments.

NI-VISA (Virtual Instrument Software Architecture) is a powerful tool that allows seamless communication with a wide range of instruments, regardless of their manufacturer. In this section, we focus on automating non-NI instruments. This practical approach not only enhances familiarity with NI-VISA functions but also provides insight into controlling external devices for a variety of applications.

Instrument Control System Overview

NI LabVIEW simplifies automating instrument-based processes by consolidating multiple instruments into a single development environment. A basic instrument control system typically includes three main components: the instrument, hardware connectivity, and a computer running software.



Instruments include amplifiers, analyzers, calibrators, oscilloscopes, power supplies, switches, function generators, and more. These instruments serve as the core hardware for measurement and control.



To communicate with the instrument, you need hardware connectivity, which includes both the physical cable and the communication protocol or bus. Common bus types include GPIB, serial, USB, and Ethernet, each offering different capabilities for latency and bandwidth. Many instruments are equipped with multiple ports, allowing flexible connectivity options to suit different system requirements.

Finally, NI LabVIEW software on the computer ties everything together. It enables you to acquire, analyze, and



visualize data from your instruments.

NI-VISA Overview

NI-VISA is an API that provides a programming interface to control Ethernet/LXI, GPIB, serial, USB, PXI, and VXI instruments in NI application development environments like NI LabVIEW, NI LabWindows/CVI, and NI Measurement Studio. The API is installed through the NI-VISA driver.



Note For Windows and Mac operating systems, NI-VISA has its own separate download ([NI-VISA Download page](#)). For Linux operating systems, NI-VISA is bundled with other NI device drivers in one single download called NI Linux Device Drivers ([NI Linux Device Drivers Download page](#)).

NI-VISA is recommended because of the following three advantages

7. **Interface Independence-** NI-VISA uses many of the same operations to communicate with instruments, regardless of the interface type. This makes it easy to switch interfaces and provides a single language to learn for multiple different instruments. For example, the VISA command to write an ASCII string to a message-based instrument is the same whether the instrument interface is serial, GPIB, or USB.
8. **Platform Portability-** NI-VISA is designed so that programs written using VISA function calls can move from one platform to another. VISA does this by defining its own data types. This ensures your application performs consistently across different platforms. Therefore, a VISA application written in NI LabVIEW can be easily ported to any platform that supports NI LabVIEW. NI-VISA supports several operating systems, including Windows OS versions, MacOS versions, and Linux distributions.



Note Check supported versions of NI-VISA with LabVIEW development systems here: [NI-VISA and LabVIEW Version Compatibility - NI](#)



Note Check supported features of the NI-VISA hardware driver with different operating systems here: [NI-VISA Features and Operating System Compatibility - NI](#)

9. **Ease of Use-** NI-VISA is an extremely easy interface to learn. It provides an API that has bus-independent functions for most of its I/O functionality. The most used functionality for instrumentation is provided in a compact command set, eliminating the need to learn low level communication protocols for multiple interface types.



Note The NI-VISA User Manual is available here: [VISA Overview - NI](#)

NI-VISA Terminology

Before you start using NI-VISA, it is helpful to learn some of the terminology commonly used throughout the development process.

- **VISA Resource** - A VISA resource is any instrument in your system. If you have multiple instruments connected to one host, each instrument is considered a VISA resource. The resource is a complete description of the set of capabilities for the instrument. The VISA resource type will depend on your instrument. There are several types: INSTR, SOCKET, RAW, etc.
- **VISA Resource Name (Instrument Descriptor)-** The VISA Resource Name is the exact name of the VISA resource. It specifies the instrument interface type, the instrument address, and the VISA Session type. The VISA resource name control is used to communicate with the instrument in your development environment. Use this information to see if your instrument is recognized and addressed properly in your program.
- **VISA Session-** A VISA session is a path of communication to a VISA Resource. A session is opened using the operation viOpen(). You must open a VISA Session any time you want to perform VISA communication to an instrument.
- **VISA Alias** - A VISA alias is the nickname for a VISA Resource. The alias provides a friendly name to easily identify an instrument.

Typical NI-VISA Application

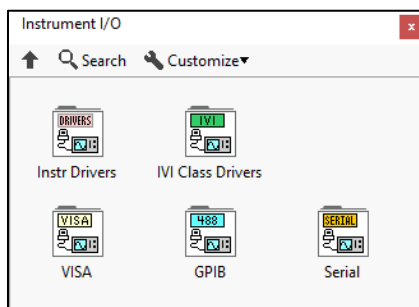
Typical VISA application would go through the following steps.

1. Open a Session to a given VISA Resource.
2. Configure communication settings on the given resource (set baud rates, termination character, etc.).
3. Perform writes and reads to the instrument.
4. Close the Session to the Resource.
5. Handle any errors that may have occurred.

You would also follow this exact same format if the instrument was serial, USB, Ethernet, or any of the other interface types VISA supports. All you would have to change is the VISA Resource Name wired to the input of the VISA Open.

NI Instrument I/O Palette

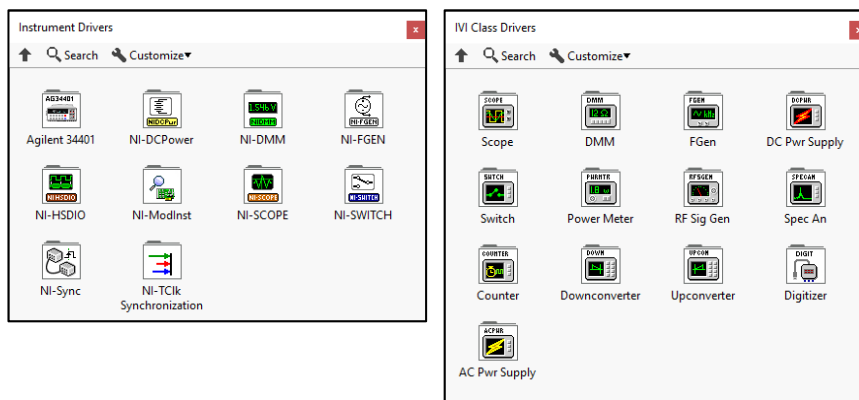
The Instrument I/O Palette in NI LabVIEW provides tools and functions used to communicate with external instruments.



Note Check Software Required for Instrument Control here: [Software Required for Instrument Control](#)

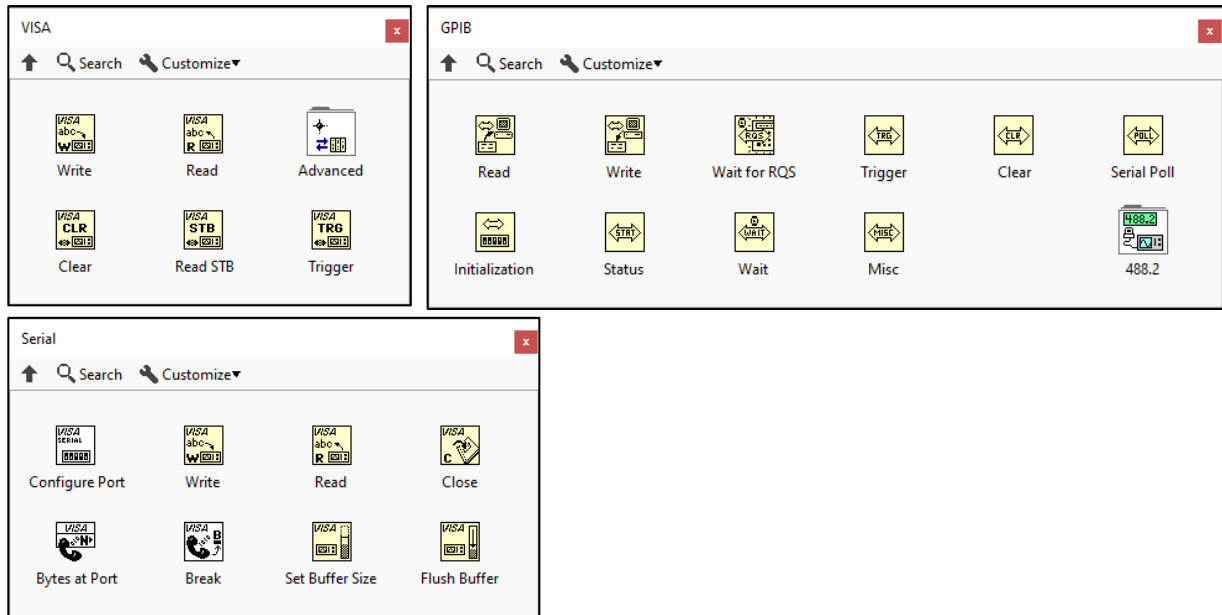
It includes the following key folders:

1. **Instr Drivers:** Provides access to instrument-specific drivers for seamless integration with supported devices, allowing you to simplify communication and control.
2. **IVI Class Drivers:** Supports Interchangeable Virtual Instruments (IVI) for standardized instrument control, enabling hardware abstraction and easier instrument replacement.



3. **VISA:** Contains functions for configuring and communicating with instruments using the VISA protocol, which supports various communication buses such as GPIB, USB, Serial, and Ethernet.
4. **GPIB:** Includes functions for interfacing with instruments over the GPIB bus, a widely used protocol for industrial instrumentation.

5. **Serial:** Contains tools for configuring and communicating with devices over serial communication protocols (RS-232, RS-485).



Tip Select **Help»Find Instrument Drivers** to display the **NI Instrument Driver Finder**. You also can select **Tools»Instrumentation»Find Instrument Drivers** or click the Connect to Instruments link on the Find Drivers and Add-ons pane of Getting Started window to display the Instrument Driver Finder.

The following table lists some important NI-VISA palette VIs along with their descriptions.

Function	Terminal	Description
VISA Open		Opens a session to the device specified by VISA resource name and returns a session identifier that can be used to call any other operations of that device. Note: For each VISA session that you open, you should close the session when you are finished with it.
VISA Write		Writes the data from write buffer to the device or interface specified by VISA resource name .
VISA Read		Reads the specified number of bytes from the device or interface specified by VISA resource name and returns the data in read buffer .
VISA Clear		Clears the input and output buffers of the device.
VISA Close		Closes a device session or event object specified by VISA resource name.

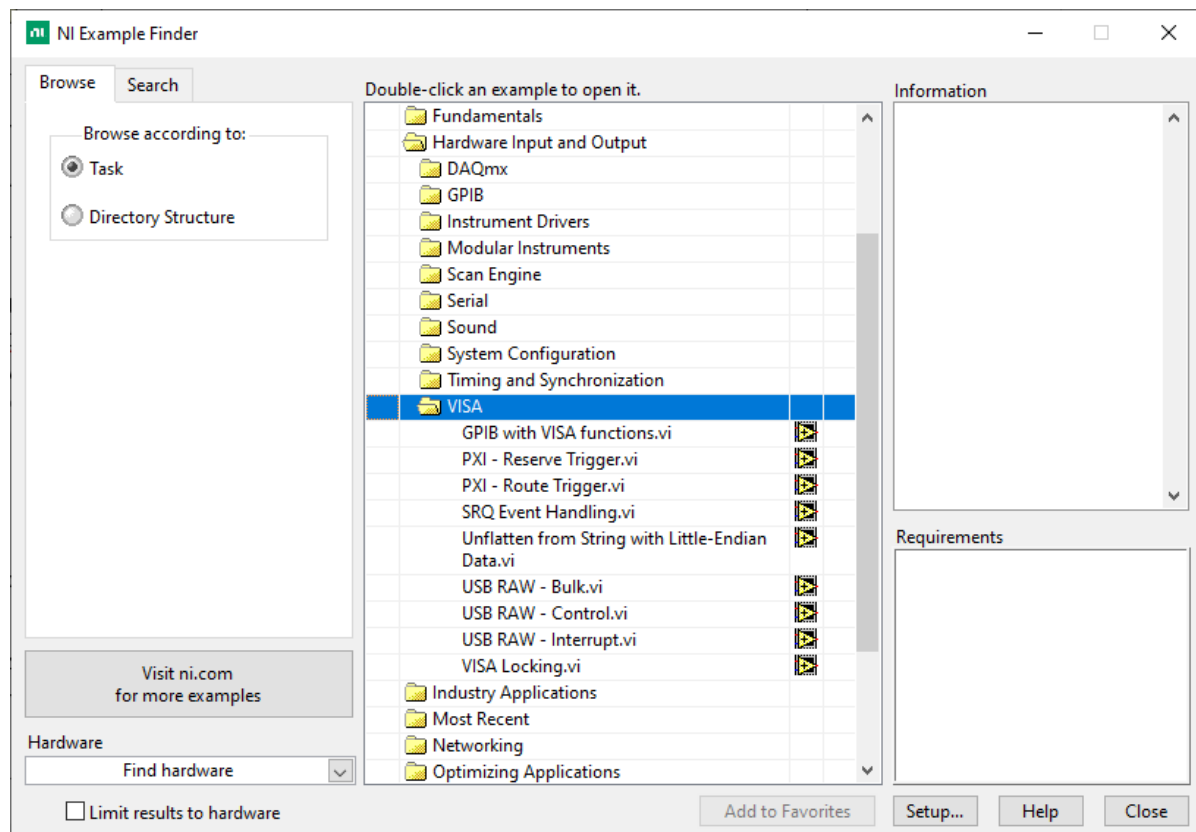
VISA Find Resource		Queries the system to locate the devices associated with a specified interface. All resource strings returned by this function are recognized by VISA Open.
VISA Configure Serial Port		Initializes the serial port specified by VISA resource name to the specified settings. Wire data to the VISA resource name input to determine the polymorphic instance to use or manually select the instance.



Tip You can use the Open VISA Session Monitor VI in the `labview\vi.lib\ Utility\visa.llb` to close all open VISA sessions. Alternatively, you could save work, exit, and re-enter LabVIEW. Exiting LabVIEW closes all open VISA sessions. You also can select the **Automatically close VISA sessions** option on the **Environment** page of the **Options** dialog box.

NI-VISA Examples

Examples for using the NI VISA are available in the NI LabVIEW Example Finder (found in Help >> Find Examples) under Hardware Input and output >> VISA. In this section, you'll also find examples for other hardware communication protocols, including Serial, GPIB, and more.

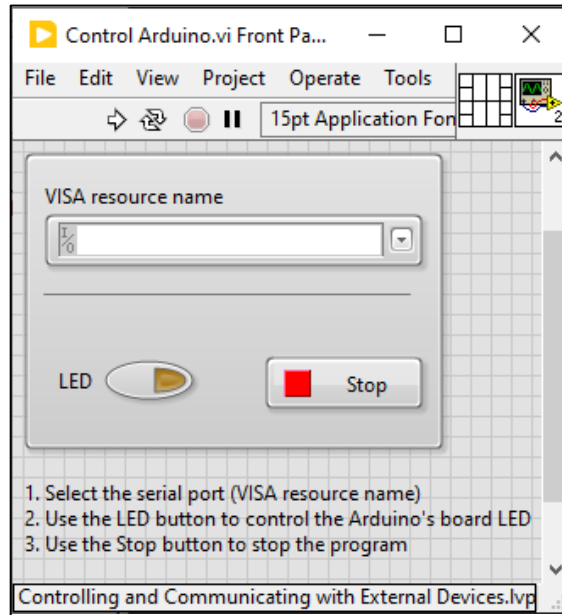


execution.

- 8 **VISA Close** — Closes the communication session with the specified VISA resource, releasing the hardware for other applications.

4. Modify the front panel shown in Figure 8.

Figure 8. Control Arduino VI Front Panel



5. Save the VI.
 6. Open Arduino IDE and create a new sketch.
- ☐ Replace the default template code with the following:

```
#define LED 13 // Define the pin number for the LED

int variable; // Variable to store incoming data

void setup() {
  Serial.begin(9600); // Set the communication baud rate to 9600
  pinMode(LED, OUTPUT); // Set pin 13 as an output for the LED
}

void loop() {
  // Check if data is available to read from the serial port
  if (Serial.available()) {
    variable = Serial.read(); // Read the incoming data

    // If the received data is 'H', turn the LED on
    if (variable == 'H') {
      digitalWrite(LED, HIGH); // Turn the LED on (HIGH)
    }
    // Otherwise, turn the LED off
    else {
      digitalWrite(LED, LOW); // Turn the LED off (LOW)
    }
  }
}
```

- ☐ Save the new sketch file and name it VISA_LED_Control.ino in the Exercises\Exercise 20-1

- ☐ Run the code to check for any errors.
- ☐ Connect the Arduino hardware to the computer.
- ☐ Select the correct port for the Arduino from the Arduino IDE settings.
- ☐ Upload the code to the Arduino hardware.

Test

4. Open the Control Arduino VI on your computer.
5. Select the correct COM port in the VI's **VISA resource name** control that corresponds to the Arduino's connection (you can find the COM port in the Arduino IDE under Tools > > Port).
6. Connect the Arduino hardware to the computer via USB cable.
7. Run VI by clicking the Run button.
8. Use the **LED** button control in the VI front panel to toggle the built-in LED on the Arduino.
9. Observe the Arduino's built-in LED (connected to pin 13). It should turn ON when the LED button is activated and turn OFF when the button is deactivated.



Note If the LED does not respond, verify the following:

- The correct COM port is selected in VI.
- The Arduino hardware is properly connected to the computer.
- The Arduino sketch has been uploaded to the Arduino board.

End of Exercise 2-1

2.3 Using Raspberry Pi or Arduino with NI LabVIEW

In this section you will learn how to use Raspberry Pi or Arduino with NI LabVIEW.

There are two primary solutions for integrating NI LabVIEW with Raspberry Pi or Arduino platforms - the NI LabVIEW Hobbyist Toolkit, and the LINX by Digilent Toolkit. Both solutions are designed to be generic hardware abstraction layers for embedded devices, rather than designed for just one specific microcontroller platform.

1. **NI LabVIEW Hobbyist Toolkit** (recommended option for NI LabVIEW 2021 and later) - Most applications looking to run NI LabVIEW program on Raspberry Pi or other embedded platforms like Arduino and BeagleBone should look into using the LabVIEW Hobbyist Toolkit. This toolkit allows you to develop LabVIEW code to deploy and run on Raspberry Pi boards up to the Raspberry Pi 4.

 **Note** Do not install the package if you are running LabVIEW 2020 Community Edition or later, as the Community Edition already includes the LabVIEW Hobbyist/LINX by NI Toolkit.

 **Note** In order to make the most of the toolkit's features and have some additional menus natively integrated within the LabVIEW environment, the versions of the toolkit and LabVIEW must match. For example, install the 2023 Q1 version of the LabVIEW Hobbyist Toolkit if using LabVIEW 2023 Q1.

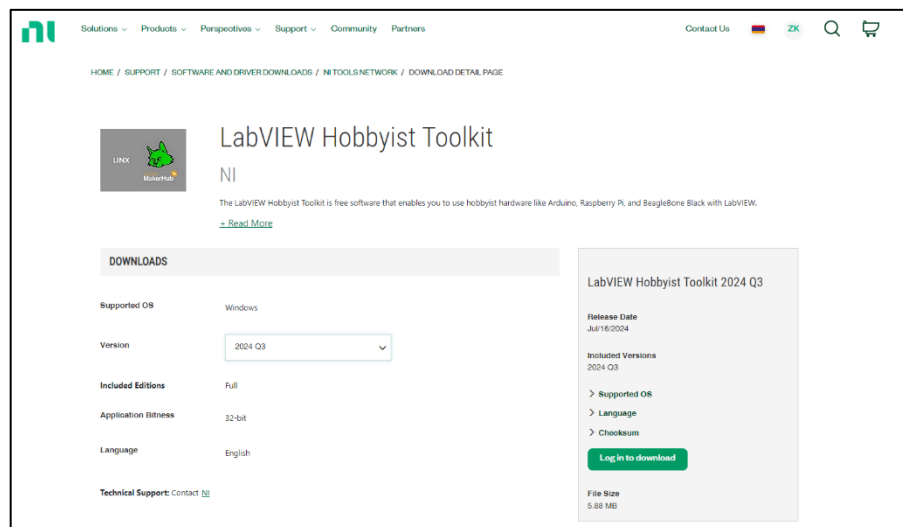
2. **LINX by Digilent Toolkit** (deprecated) - For applications running on versions older than NI LabVIEW 2020 (NI LabVIEW 2011 minimum), the third-party LINX by Digilent Toolkit for LabVIEW is the former option for designing programs for Raspberry Pi and other embedded platform devices. This solution is designed to work primarily with a host computer running a program and communicating with an embedded platform board, and only supports deploying code to run heedlessly on a Raspberry Pi up to the Raspberry Pi 3.

 **Note** The LINX by Digilent Toolkit can be downloaded through the VI Package Manager (VIPM) or from the VIPM page. VIPM is not a NI software, and any bugs or issues with VIPM are best supported by the VIPM developers.

Installing NI LabVIEW Hobbyist Toolkit

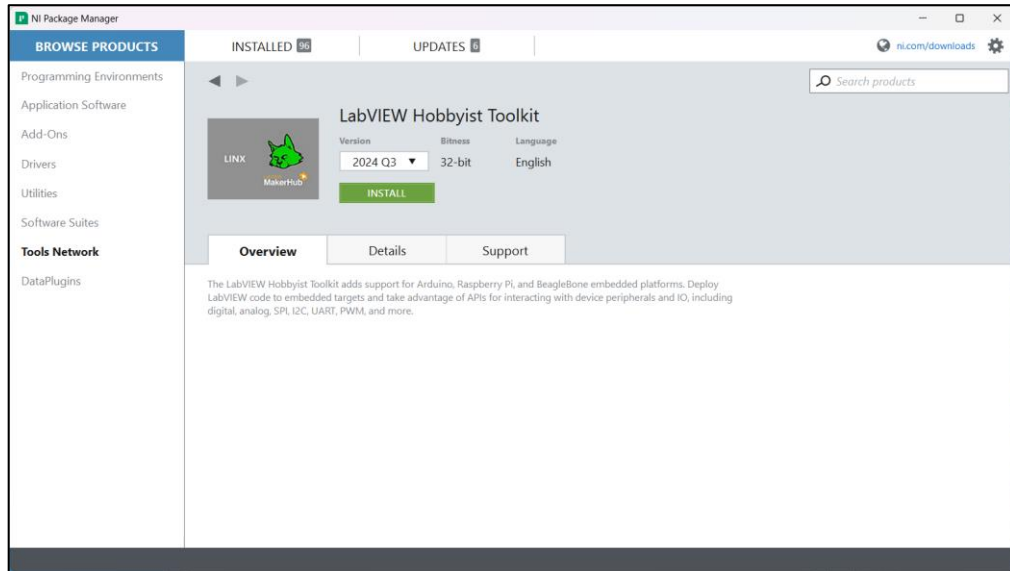
There are two options to install the LabVIEW Hobbyist Toolkit, depending on your preferences:

1. **Download from the NI Website:** Download the [LabVIEW Hobbyist Toolkit](#) that matches your NI LabVIEW version (2021 or later).



 **Note** You will need to log in to your NI account on NI.com to download the toolkit or create an account if you don't have one.

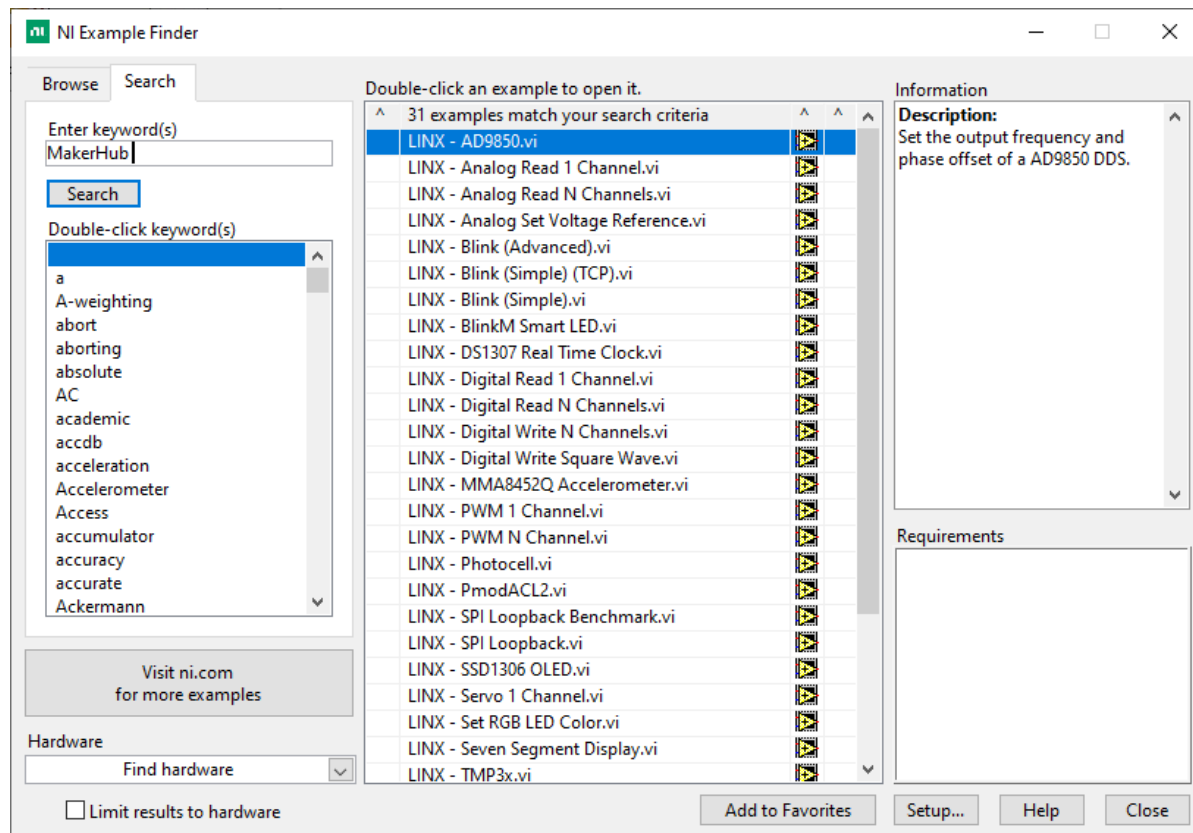
2. **Use the NI Package Manager:** The toolkit can also be found and directly downloaded using the NI Package Manager.



Note The toolkit is available only for the 32-bit version of NI LabVIEW.

NI LabVIEW Hobbyist Toolkit Examples

There are multiple examples from NI LabVIEW that demonstrate how to set up your hardware and use NI LabVIEW Hobbyist Toolkit. Examples are available in the NI LabVIEW Example Finder (found in Help >> Find Examples). by searching with the LINX or MakerHub keywords.





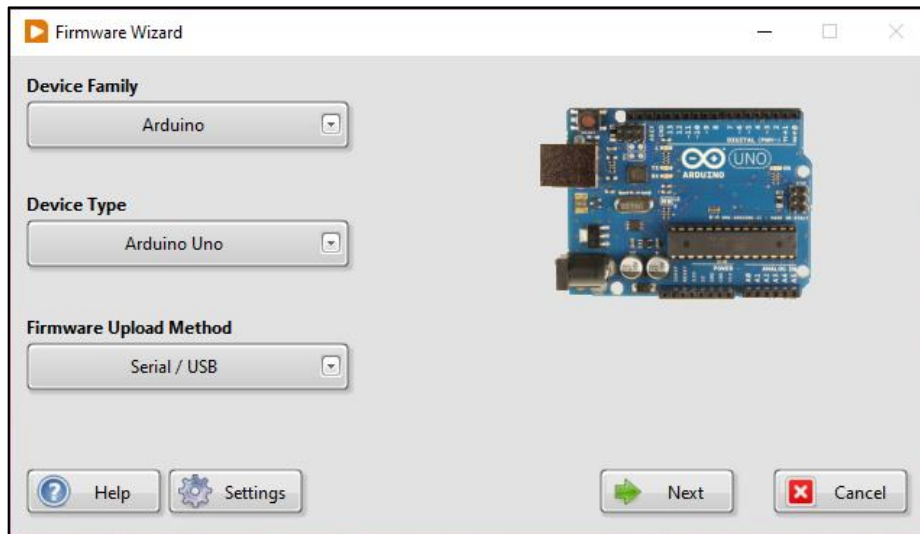
Activity 2-1: Using the NI LabVIEW Hobbyist Toolkit to Run NI LabVIEW Programs on Arduino

Goal

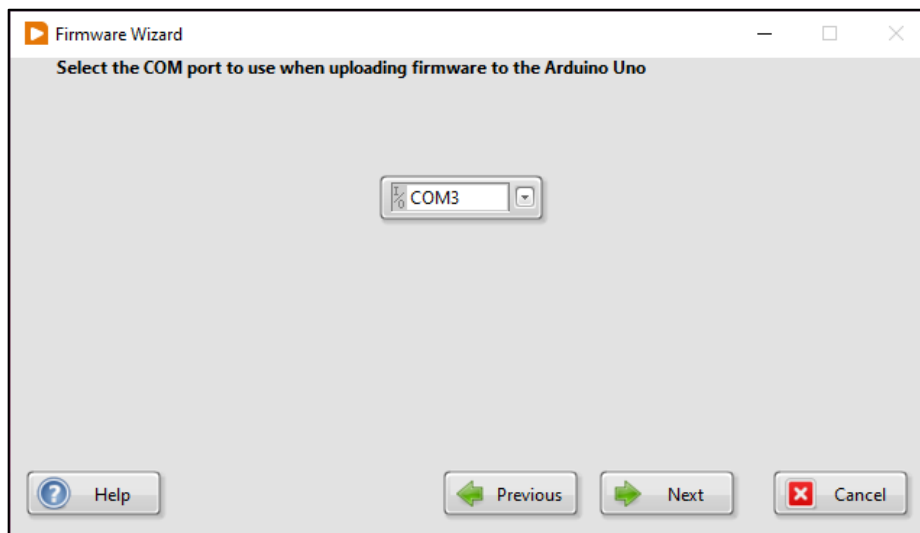
The goal of this activity is to learn how to use the NI LabVIEW Hobbyist Toolkit to program and control an Arduino board.

Instructions

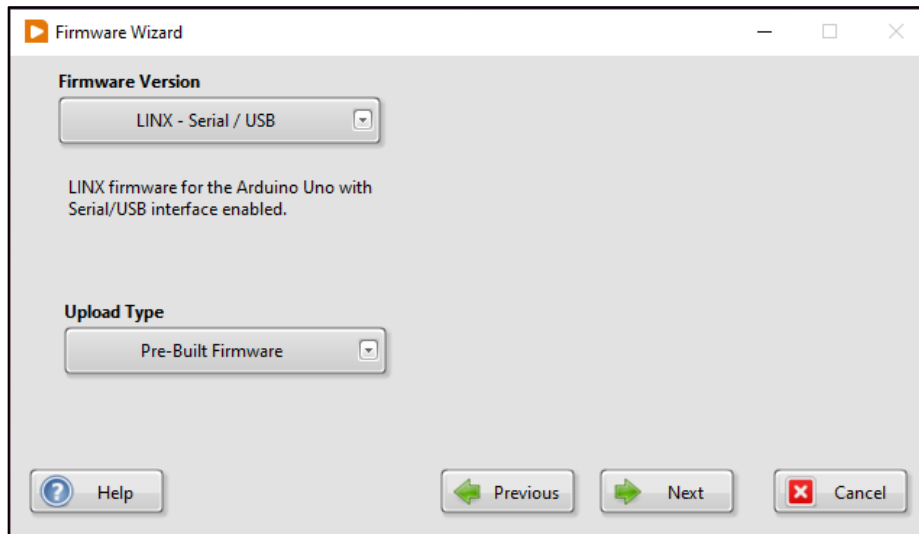
7. Launch NI LabVIEW. Select Tools » Hobbyist » Firmware Wizard... to start the NI LabVIEW Hobbyist Toolkit Firmware Wizard. This tool helps to configure the third-party hardware and upload toolkit firmware.
8. From the Device Family, choose Arduino. Select your Arduino device type and firmware upload method, then click **Next**.



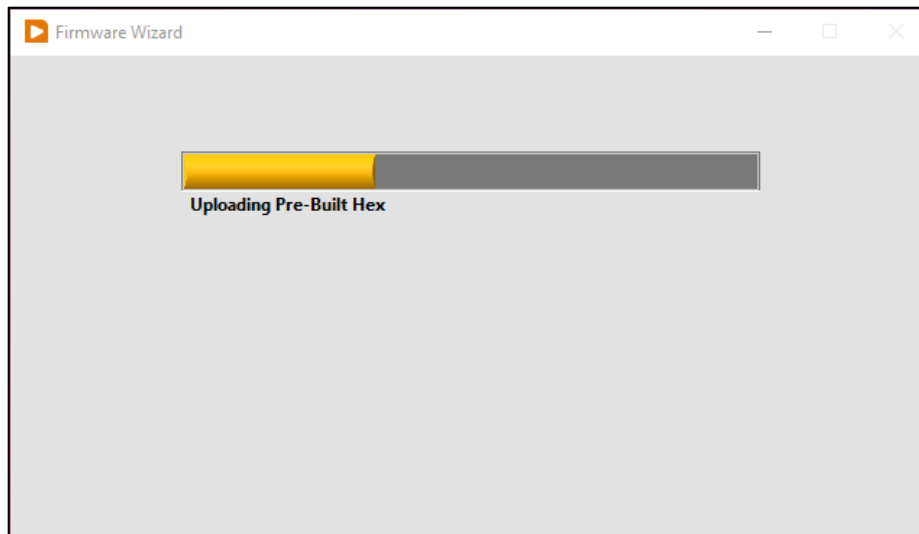
9. Select the COM port to use when uploading the firmware to the Arduino hardware. Then click **Next**.



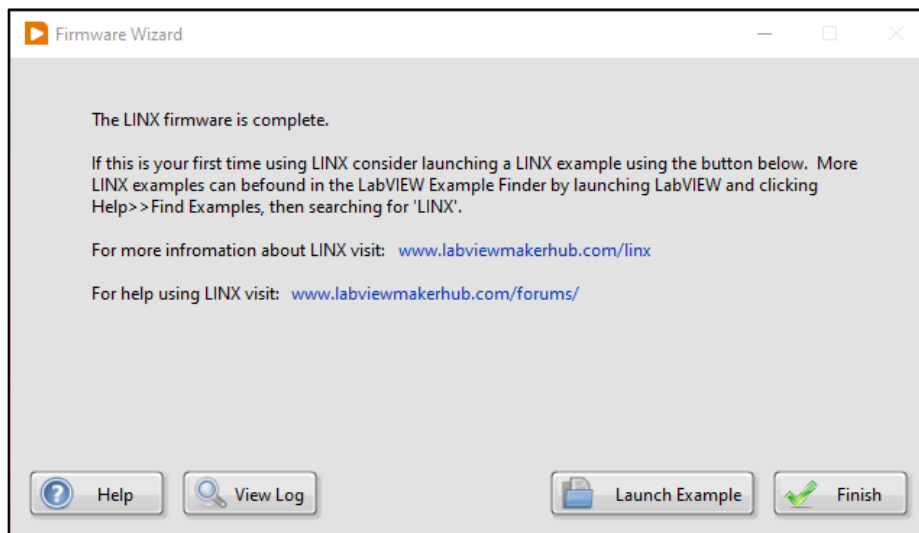
10. Choose the upload type and firmware version, then click **Next**.



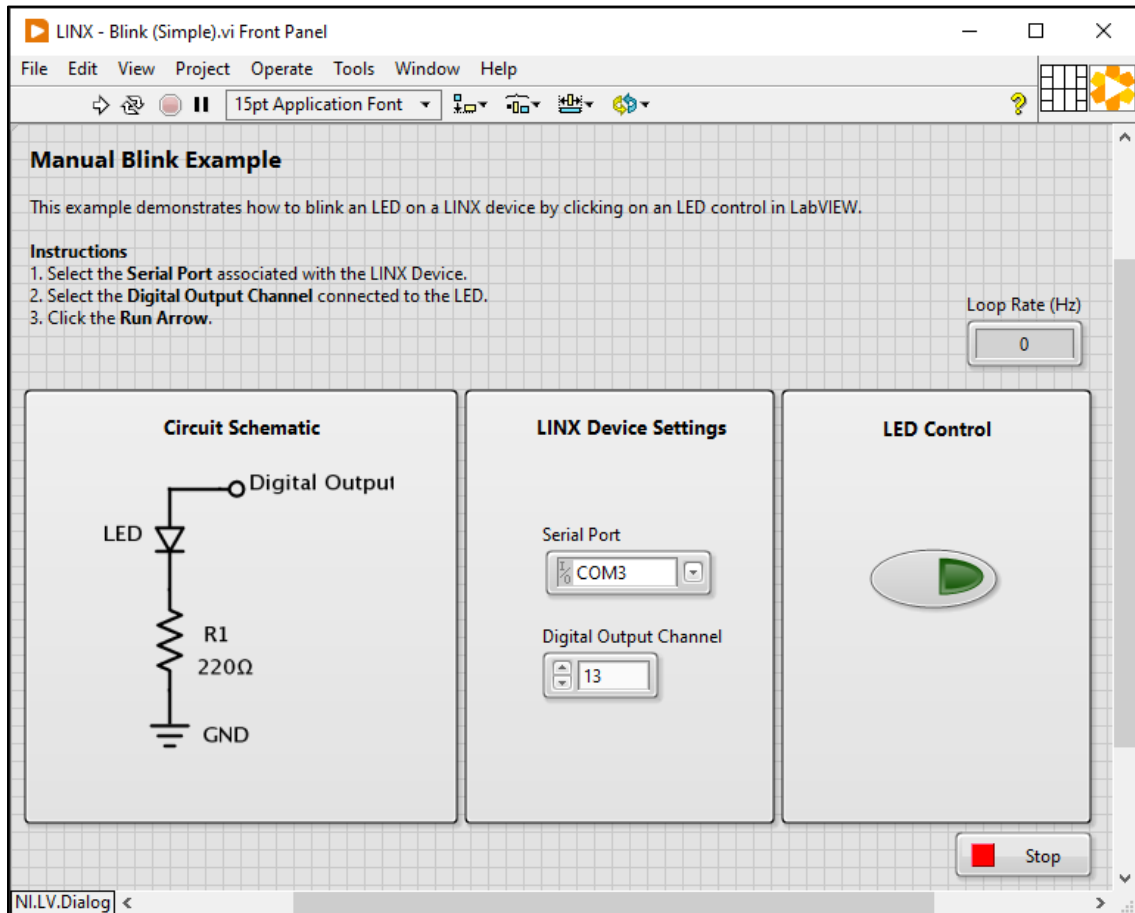
11. Wait for the firmware to upload.



12. If the firmware uploads successfully, you will see the following window. Click the **Launch Example** button to open the Blink example.



13. Select the serial port associated with the Arduino hardware and run the VI. Click the LED Control button and check if it works correctly. If it does, stop the VI



14. Next, open the Block Diagram of the VI, use Context Help, and investigate the Hobbyist Toolkit VIs.



Activity 2-2: Lesson Review

7. Which of the following interfaces is supported by NI-VISA for instrument communication?
 - a. Ethernet/LXI
 - b. GPIB
 - c. Serial
 - d. USB

8. Which of the following is the correct workflow when working with NI-VISA?
 - a. Open a session, perform writes and reads, handle errors, configure communication settings, and close the session.
 - b. Open a session, configure communication settings, perform writes and reads, close the session and handle errors.
 - c. Configure communication settings, open a session, perform writes and reads, handle errors, and close the session.
 - d. Open a session, handle errors, configure communication settings, perform writes and reads, and close the session.

9. Which of the following embedded platforms is supported by the LabVIEW Hobbyist Toolkit?
 - a. Arduino
 - b. Raspberry Pi
 - c. BeagleBone
 - d. All of the above



Activity 2-2: Lesson Review - Answers

1. Which of the following interfaces is supported by NI-VISA for instrument communication?
 - a. **Ethernet/LXI**
 - b. **GPIB**
 - c. **Serial**
 - d. **USB**

2. Which of the following is the correct workflow when working with NI-VISA?
 - a. Open a session, perform writes and reads, handle errors, configure communication settings, and close the session.
 - b. **Open a session, configure communication settings, perform writes and reads, close the session and handle errors.**
 - c. Configure communication settings, open a session, perform writes and reads, handle errors, and close the session.
 - d. Open a session, handle errors, configure communication settings, perform writes and reads, and close the session.

3. Which of the following embedded platforms is supported by the LabVIEW Hobbyist Toolkit?
 - a. Arduino
 - b. Raspberry Pi
 - c. BeagleBone
 - d. **All of the above**